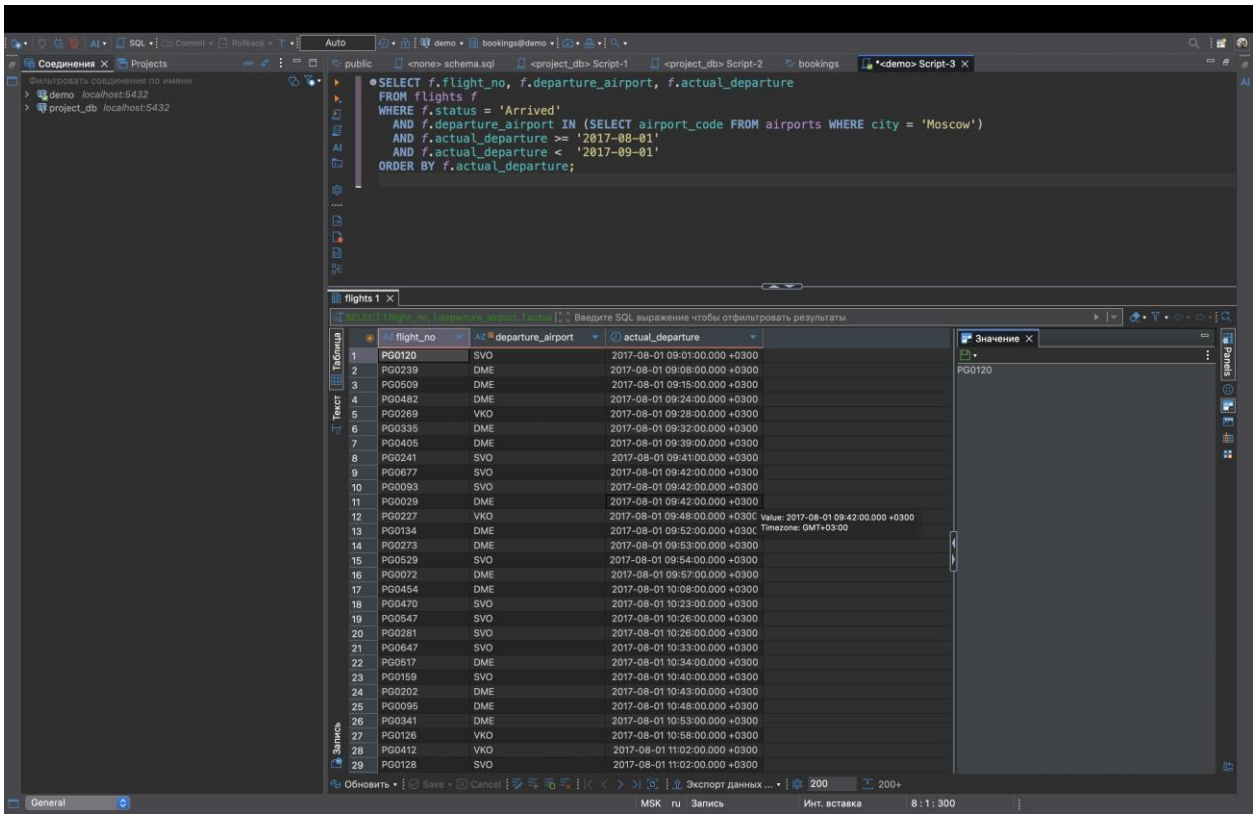
Домашнее задание №5 БД

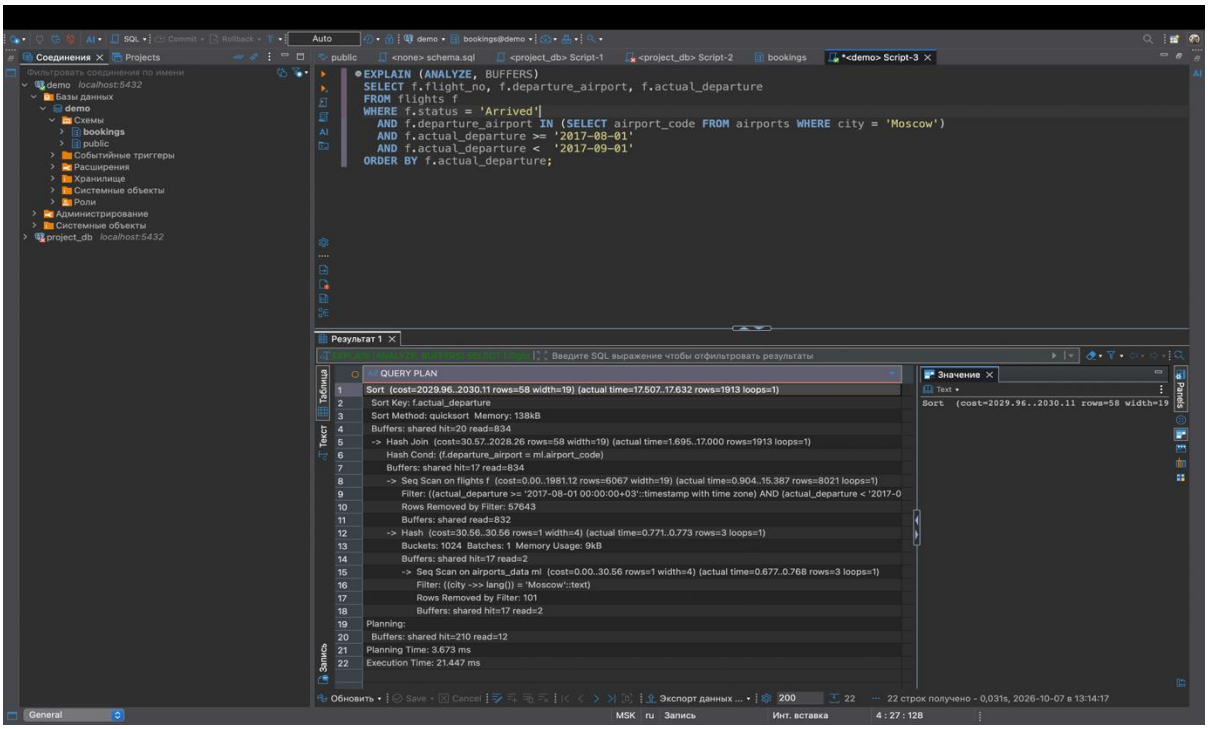
Задача 1.1. Выполненные рейсы из московских аэропортов за август 2017:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_no, f.departure_airport, f.actual_departure
FROM flights f
WHERE f.status = 'Arrived'
  AND f.departure_airport IN (SELECT airport_code FROM airports WHERE city = 'Moscow')
  AND f.actual_departure >= '2017-08-01'
  AND f.actual_departure <  '2017-09-01'
ORDER BY f.actual_departure;
```

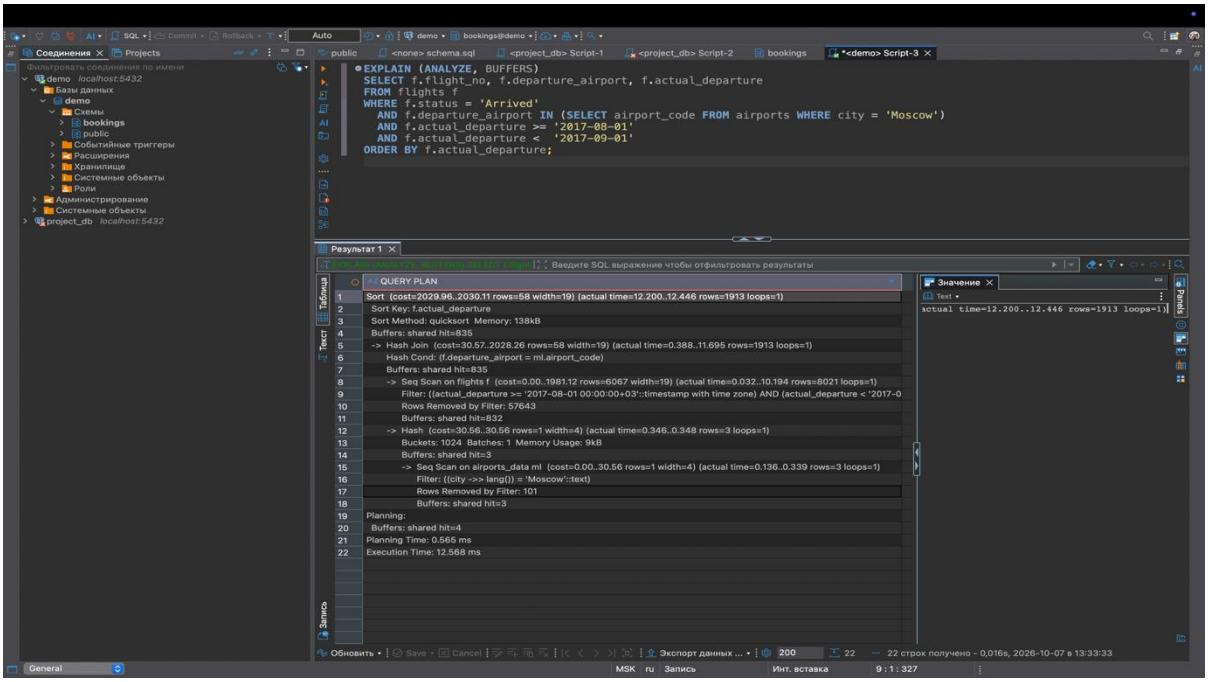
ПРОСТОЙ ЗАПУСК



1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



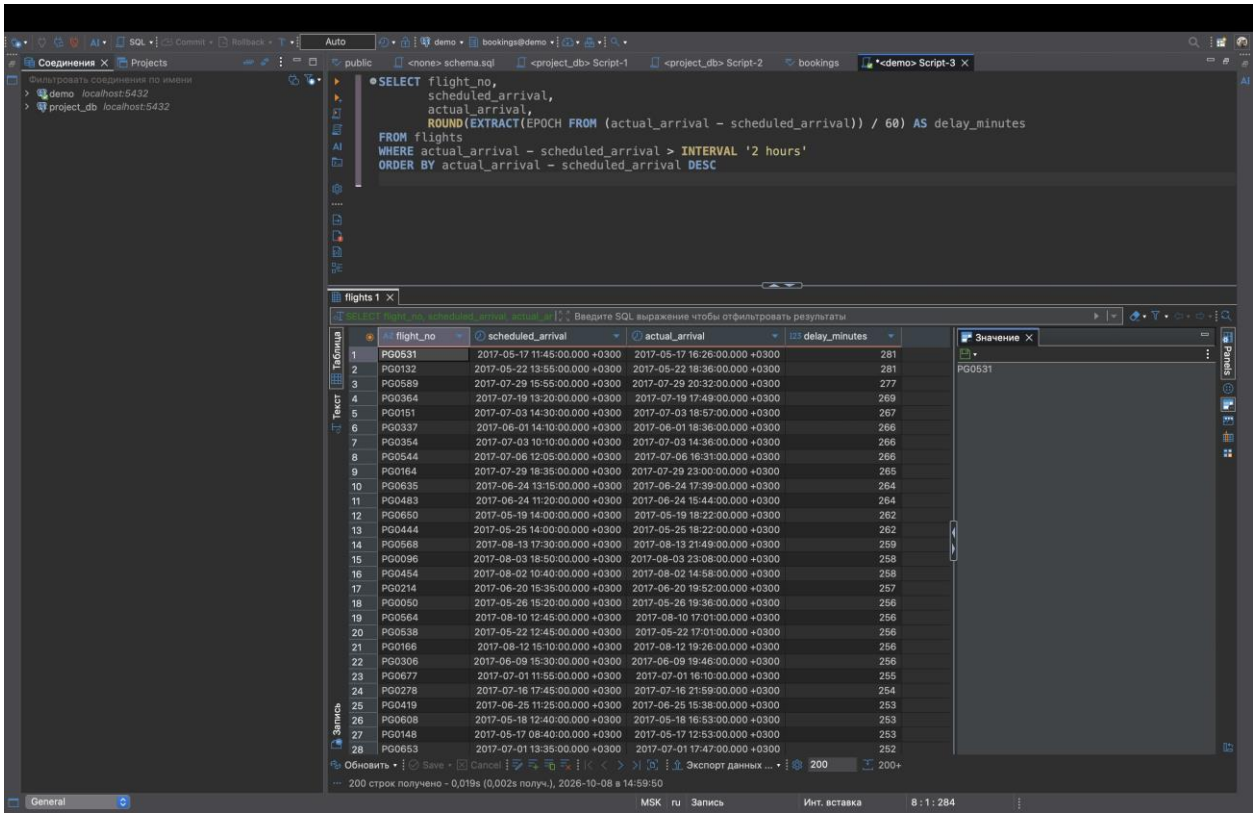
Пояснение:

При первом запуске запрос работал с диском: таблица flights целиком читалась с диска (shared read=832, всего read=834), в кеше нашлась лишь небольшая часть страниц (hit=20). При повторном запуске все 835 страниц уже лежали в кеше (shared hit=835, read нет), и время выполнения упало с 21.4 до 12.6 мс, а планирования с 3.7 до 0.6 мс. Индексы не использовались: flights и airports_data читались полным Seq Scan, соединение выполнено через Hash Join, сортировка прошла в памяти (quicksort, 138 kB)

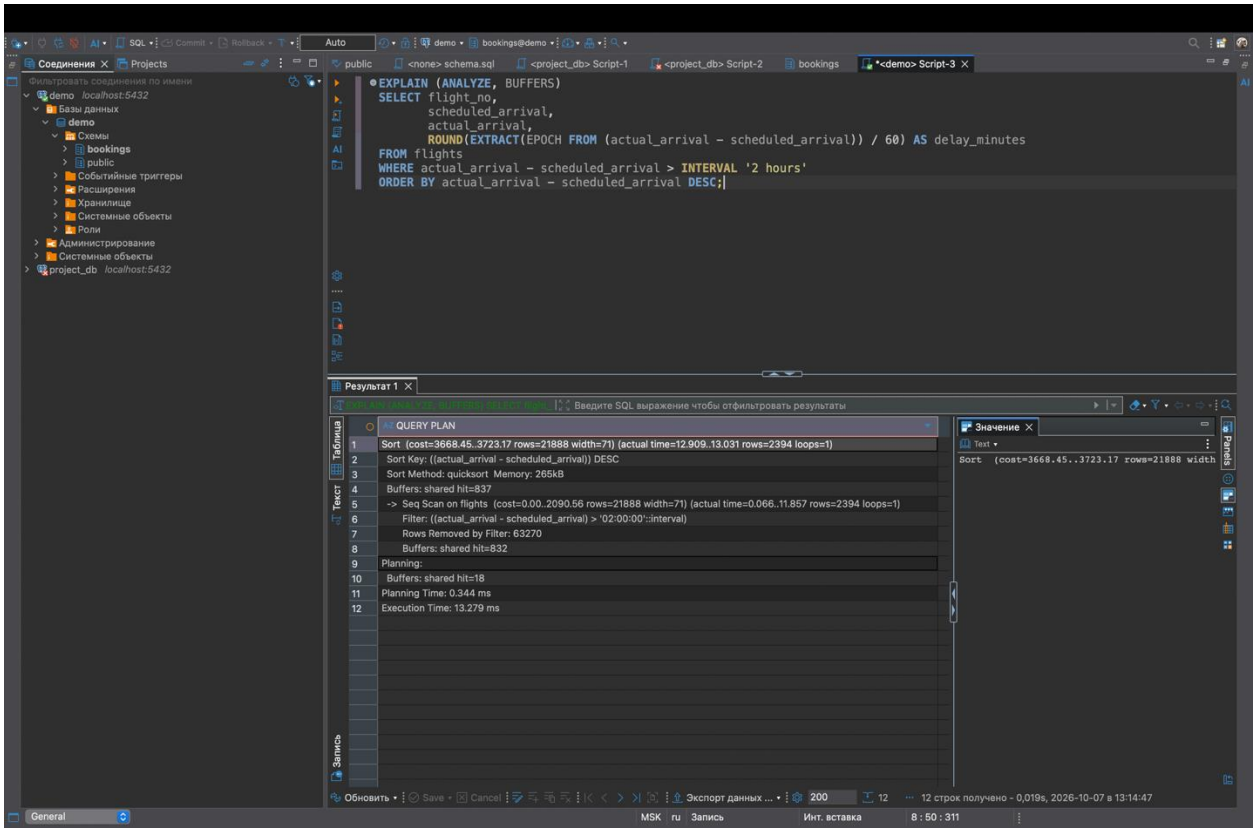
Задача 1.2. Опоздание прибытия больше 2 часов:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT flight_no,
    scheduled_arrival,
    actual_arrival,
    ROUND(EXTRACT(EPOCH FROM (actual_arrival - scheduled_arrival)) / 60) AS delay_minutes
FROM flights
WHERE actual_arrival - scheduled_arrival > INTERVAL '2 hours'
ORDER BY actual_arrival - scheduled_arrival DESC
```

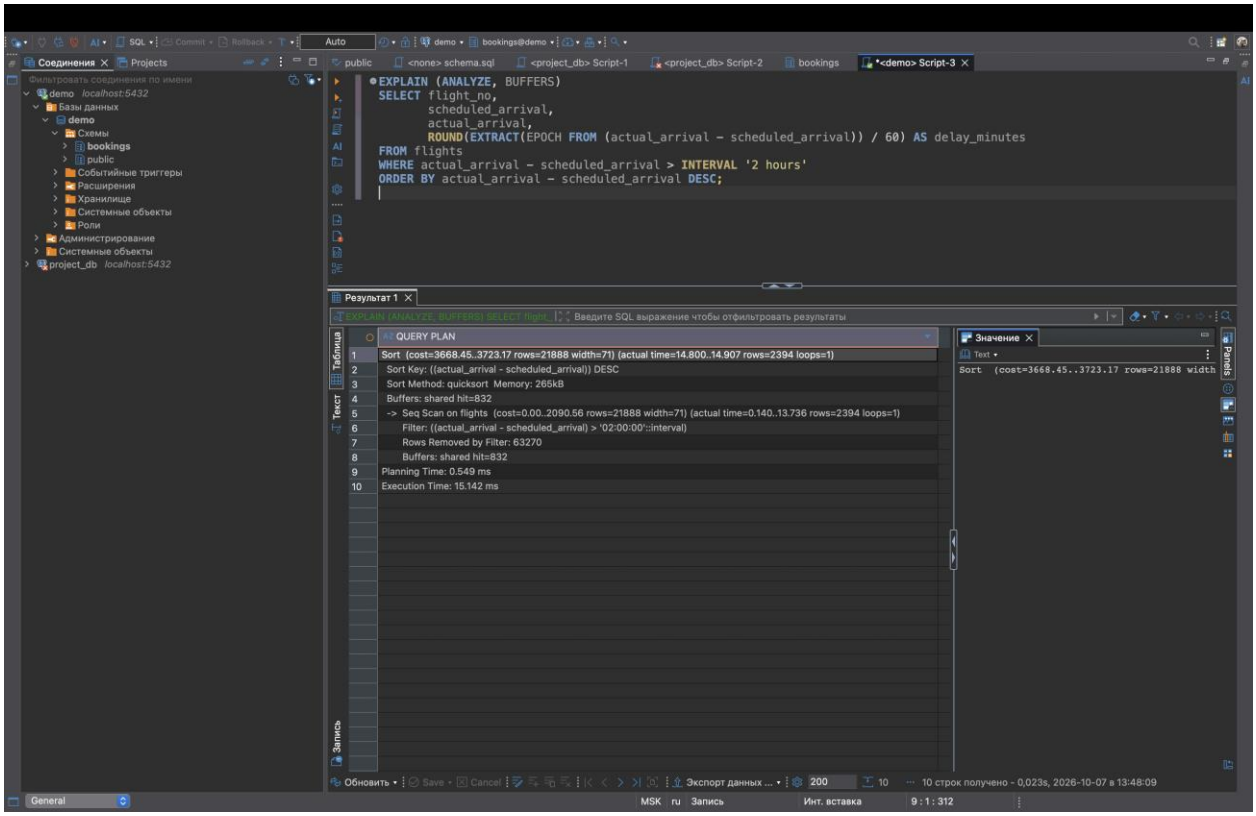
ПРОСТОЙ ЗАПУСК



1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



Пояснение:

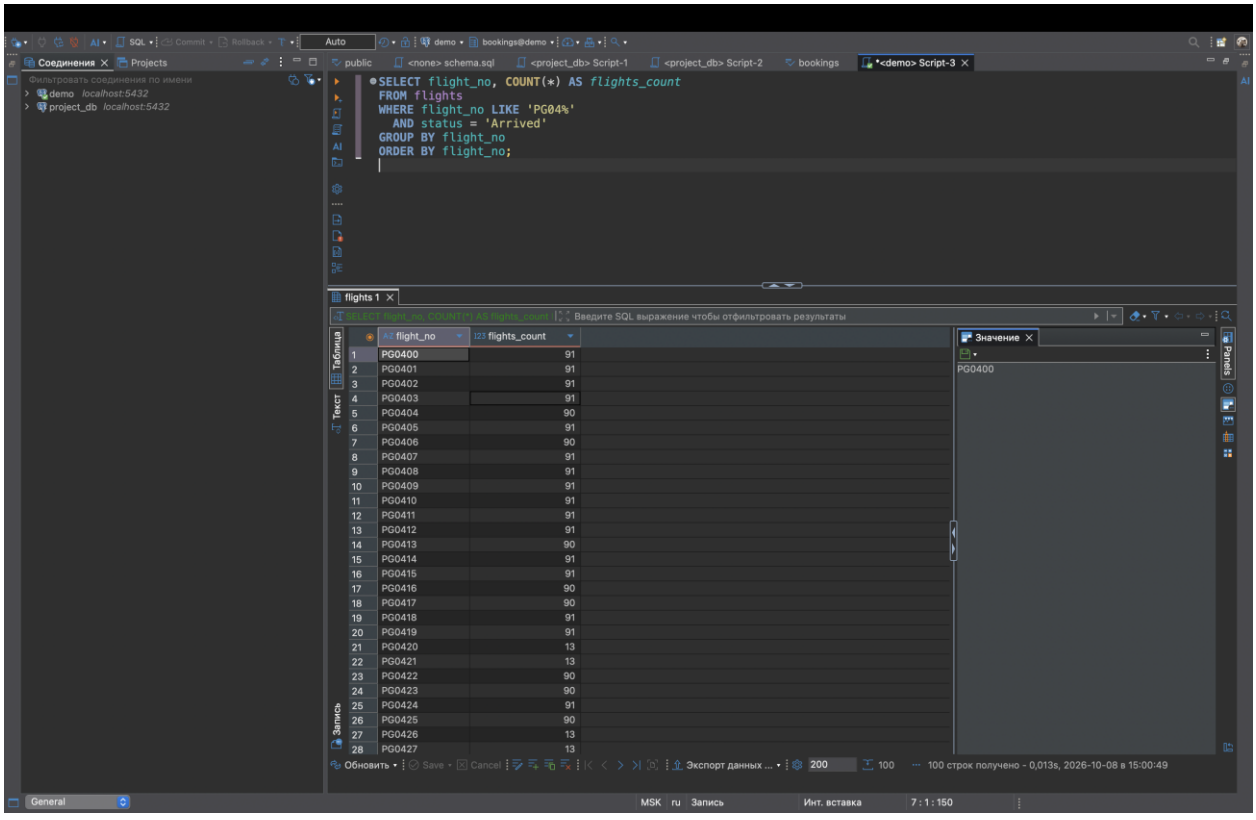
Запрос не обращался к диску: при обоих запусках в Buffers только shared hit (837 при первом запуске и 832 при втором), read нет, так как таблица flights уже находилась в кеше после предыдущих запросов. Индекс не использовался: выполнен полный Seq Scan по flights, из 65 664 строк условие по опозданию отсеяло 63 270, осталось 2394 строки

(планировщик ожидал 21 888). Сортировка по убыванию опоздания прошла в памяти (quicksort, 265 kB). Время выполнения 13.3 мс при первом запуске и 15.1 мс при втором, разница находится в пределах погрешности.

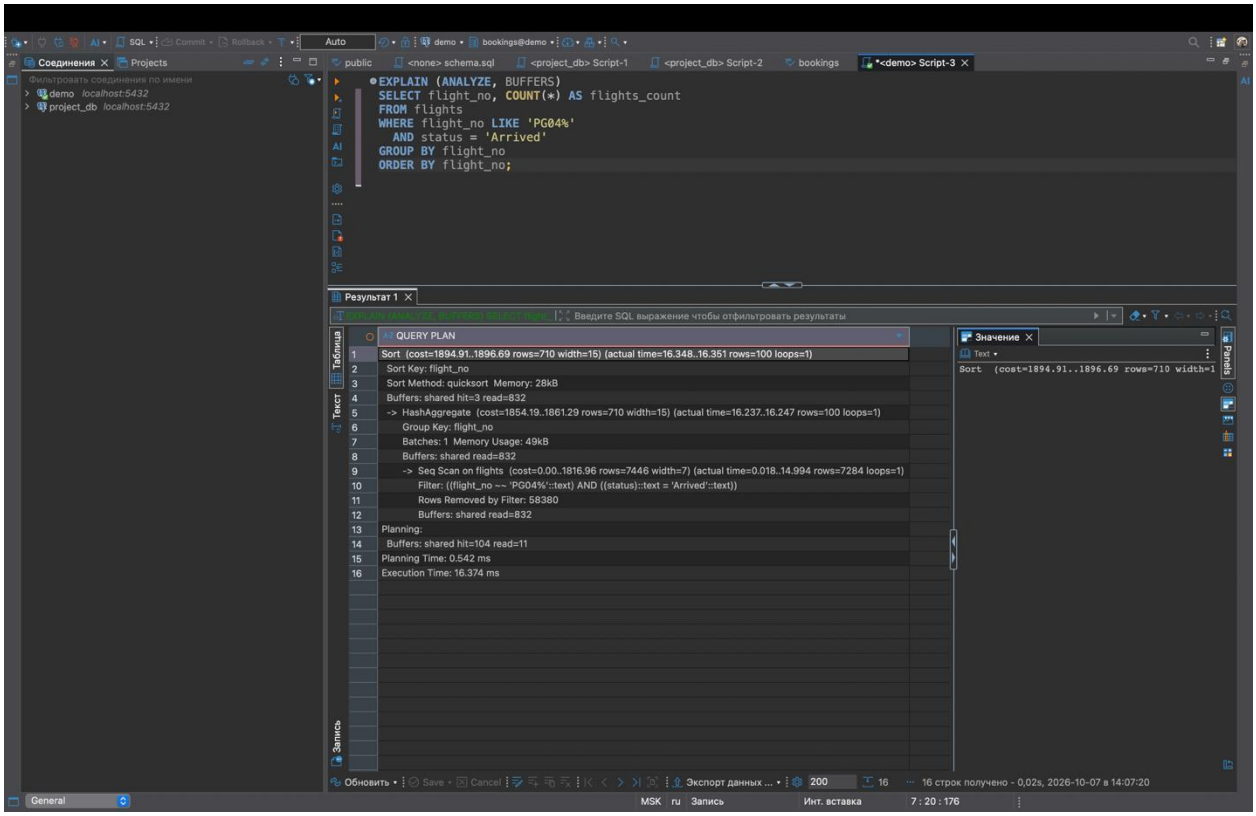
Задача 1.3. Количество выполненных рейсов по каждому номеру PG04xx:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT flight_no, COUNT(*) AS flights_count
FROM flights
WHERE flight_no LIKE 'PG04%'
  AND status = 'Arrived'
GROUP BY flight_no
ORDER BY flight_no;
```

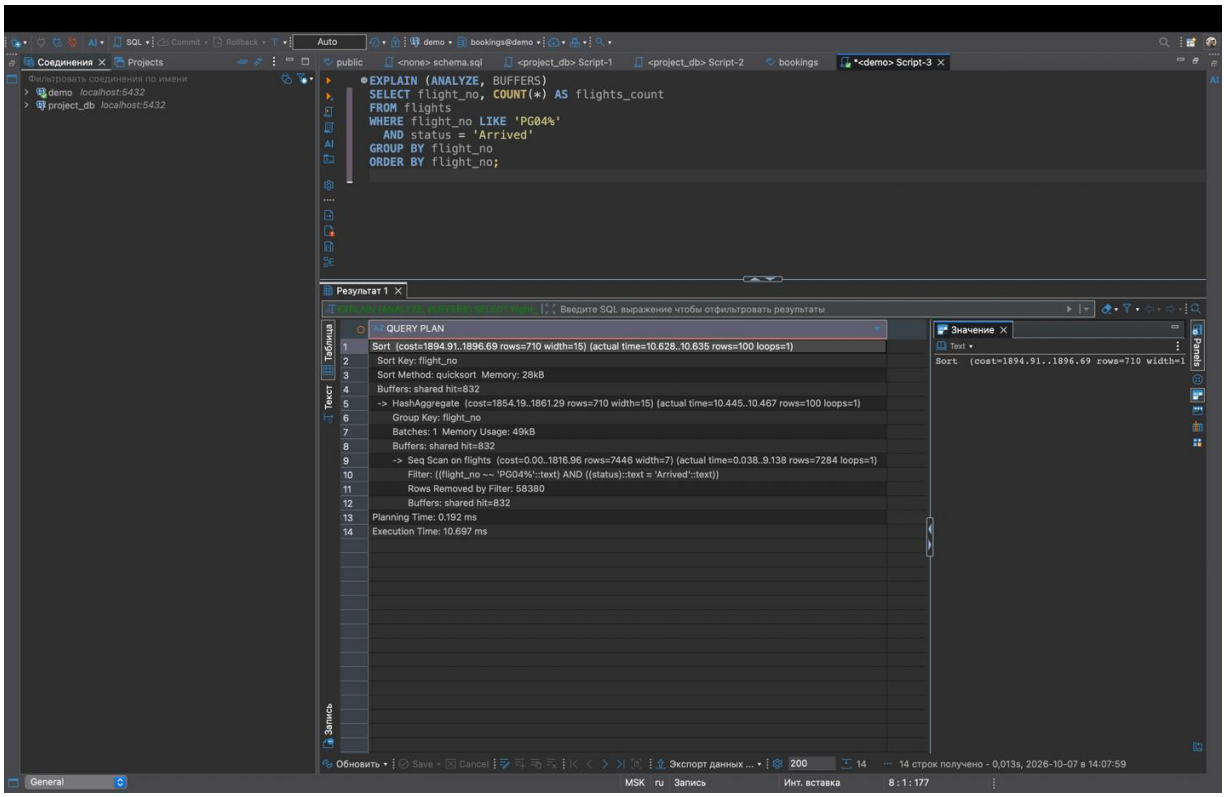
ПРОСТОЙ ЗАПУСК



1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



Пояснение:

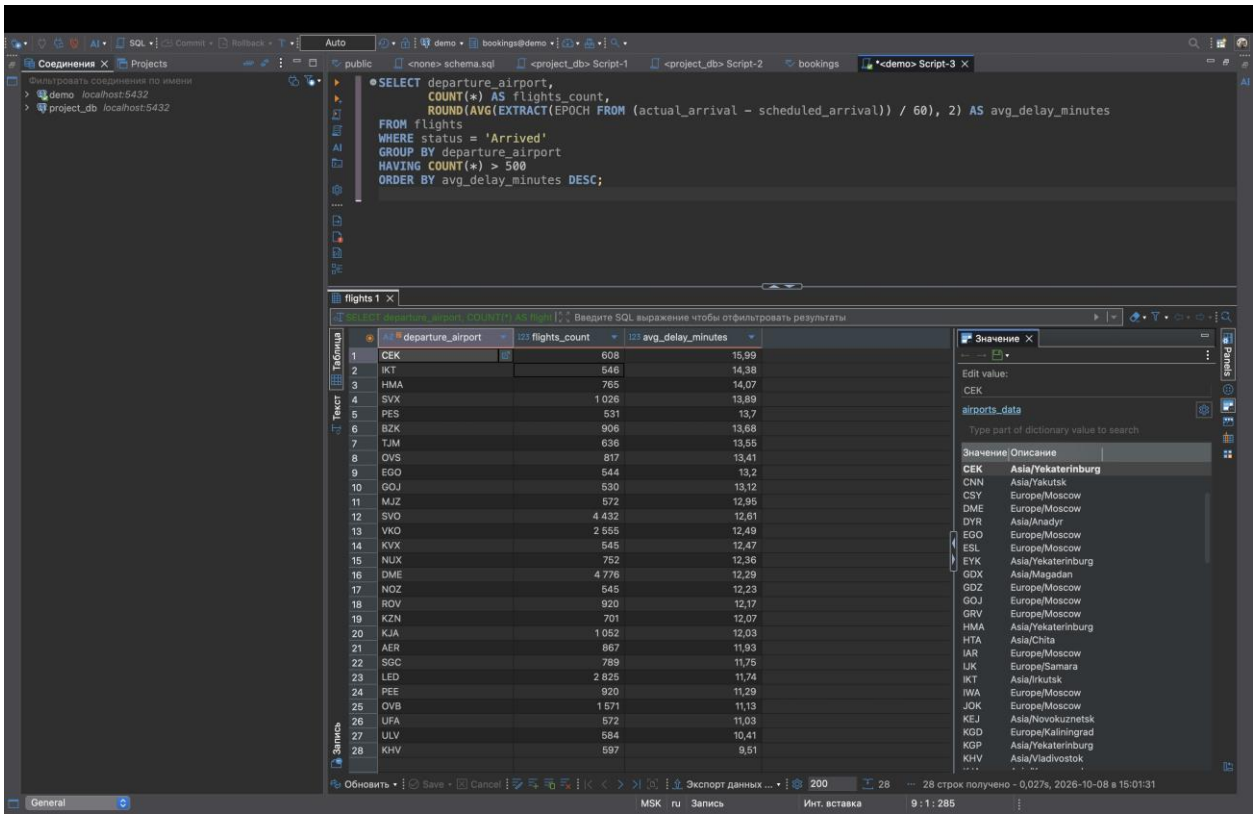
При первом запуске (14:07:20) запрос работал с диском: таблица flights читалась целиком с диска (shared read=832, в кеше только hit=3), а при втором запуске (14:07:59) все 832 страницы уже были в кеше (shared hit=832, read нет). Из-за этого время выполнения снизилось с 16.4 до 10.7 мс, а планирования с 0.54 до 0.19 мс. Индекс не использовался: условия flight_no LIKE 'PG04%' и status = 'Arrived' проверялись полным Seq Scan

(получено 7284 строки, отброшено 58 380), затем HashAggregate собрал 100 номеров рейсов. Сортировка прошла в памяти (quicksort, 28 kB).

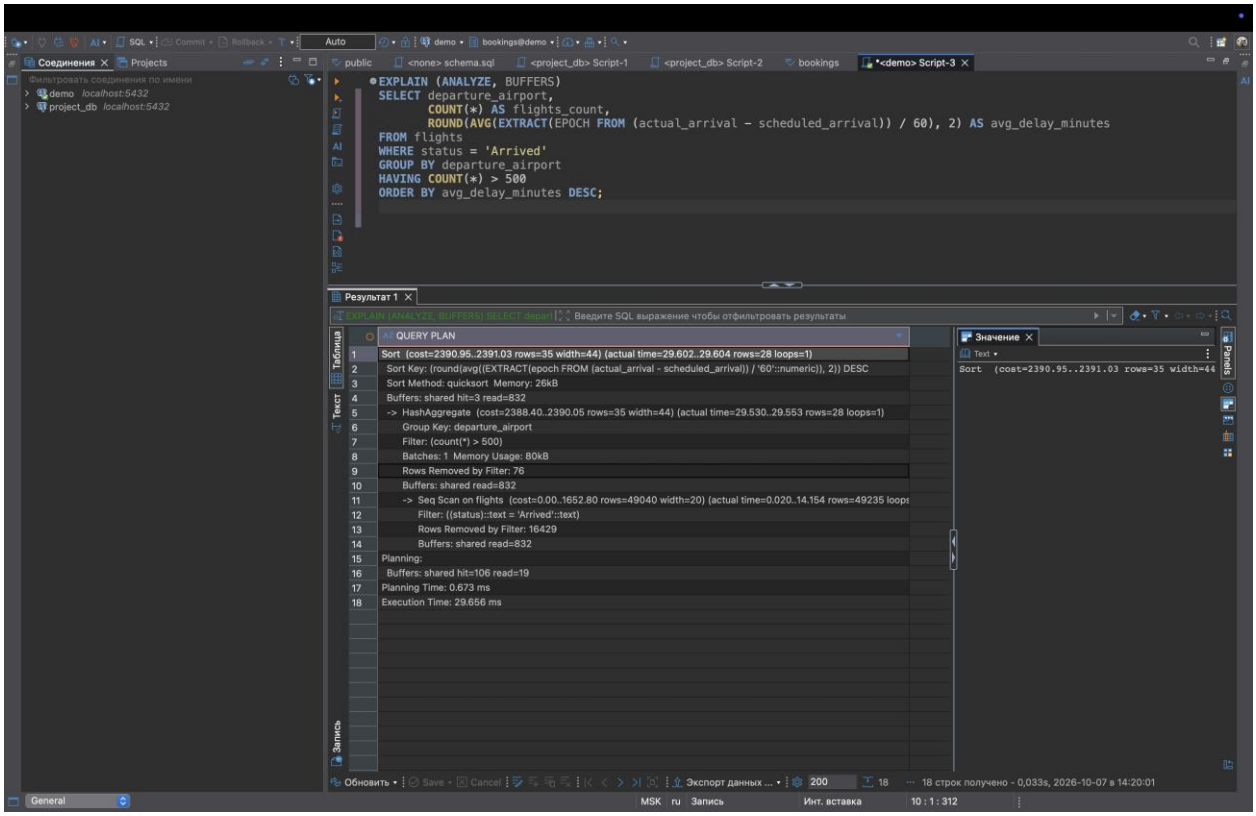
Задача 2.1. Аэропорты вылета: число рейсов и среднее опоздание прибытия:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT departure_airport,
    COUNT(*) AS flights_count,
    ROUND(AVG(EXTRACT(EPOCH FROM (actual_arrival - scheduled_arrival)) / 60), 2) AS
avg_delay_minutes
FROM flights
WHERE status = 'Arrived'
GROUP BY departure_airport
HAVING COUNT(*) > 500
ORDER BY avg_delay_minutes DESC;
```

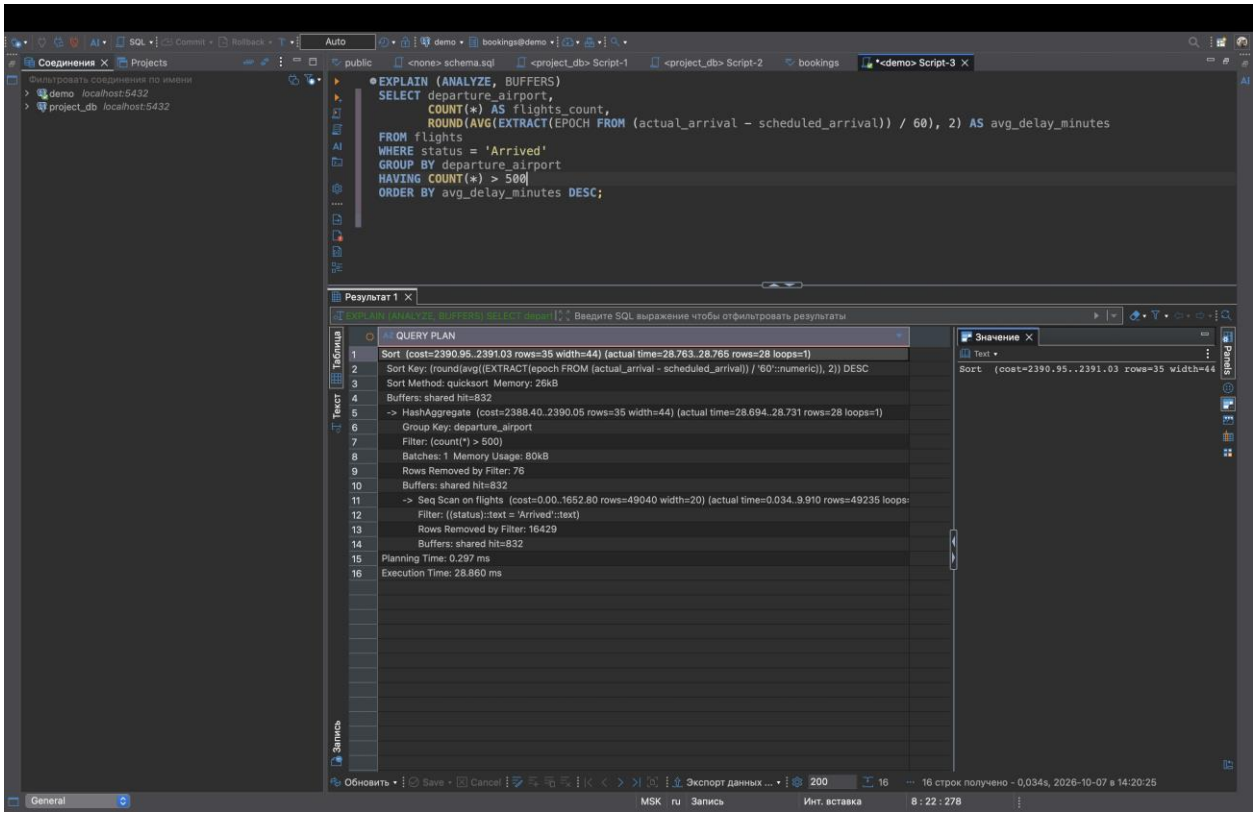
ПРОСТОЙ ЗАПУСК



1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



Пояснение

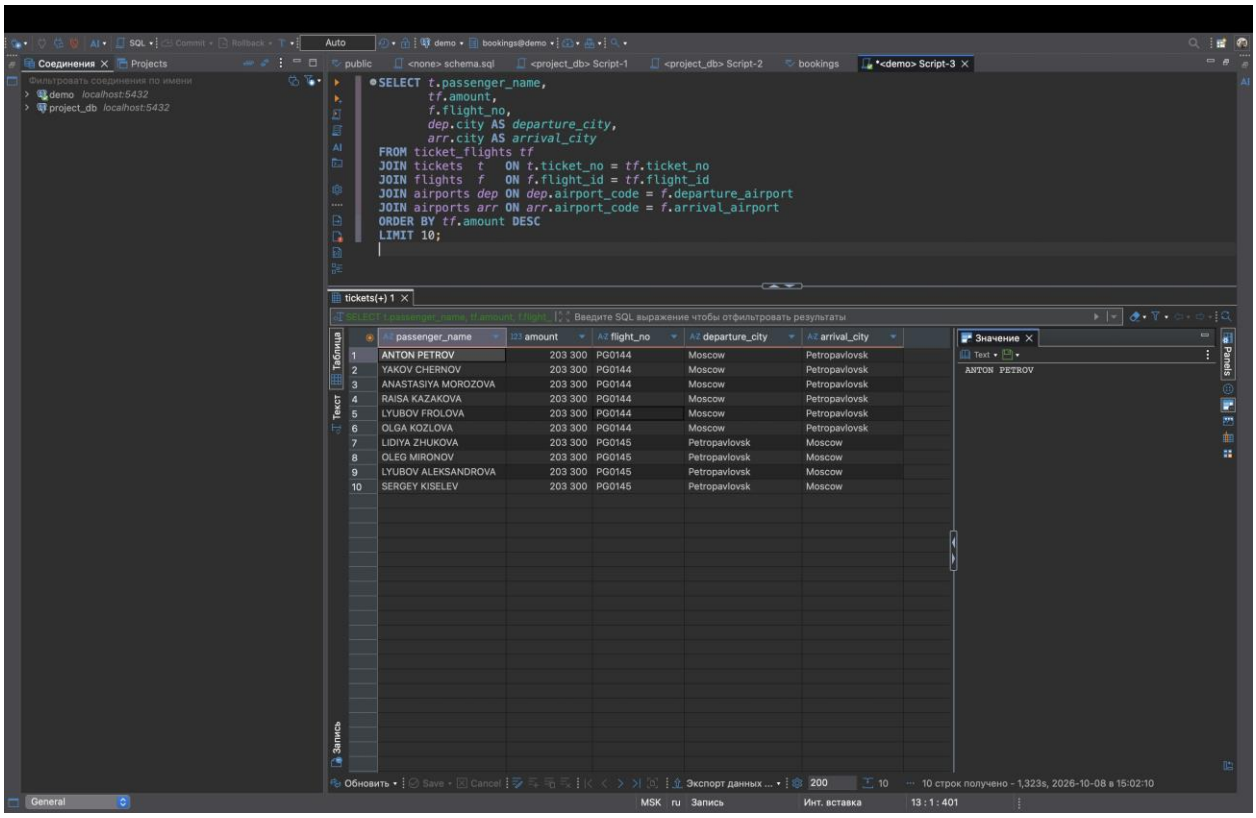
При первом запуске (14:20:01) запрос работал с диском: таблица flights читалась с диска (shared read=832, в кеше только hit=3), а при втором запуске (14:20:25) все 832 страницы уже лежали в кеше (shared hit=832, read нет). Время планирования снизилось с 0.67 до 0.30

мс, а выполнения почти не изменилось (29.7 и 28.9 мс), потому что основное время уходит на вычисление агрегатов по 49 235 строкам, а не на чтение страниц. Индекс не использовался: выполнен полный Seq Scan по flights с фильтром status = 'Arrived' (отброшено 16 429 строк), затем HashAggregate по departure_airport, где условие HAVING count(*) > 500 отсекло 76 групп и оставило 28 аэропортов. Агрегация и сортировка (quicksort, 26 kB) прошли в памяти, временные файлы не использовались.

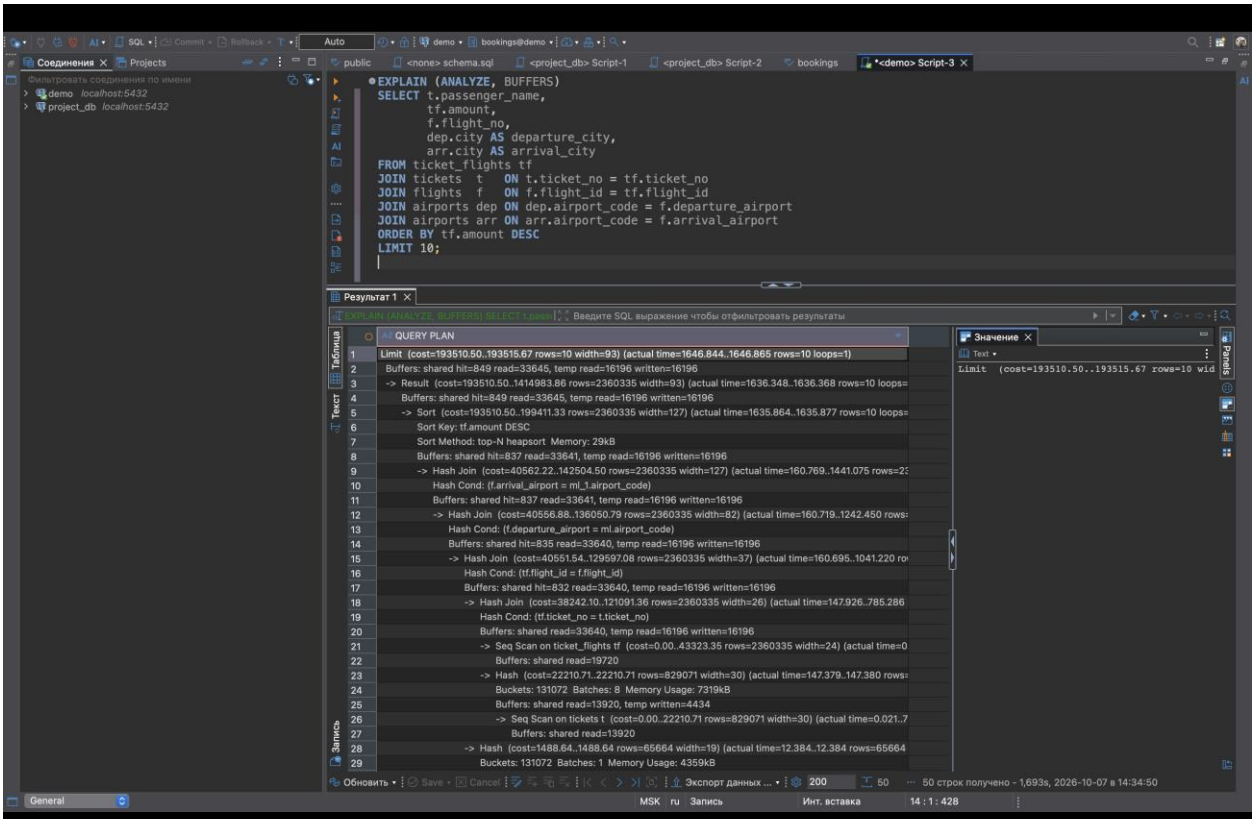
Задача 3.1. Топ-10 самых дорогих проданных билетов:

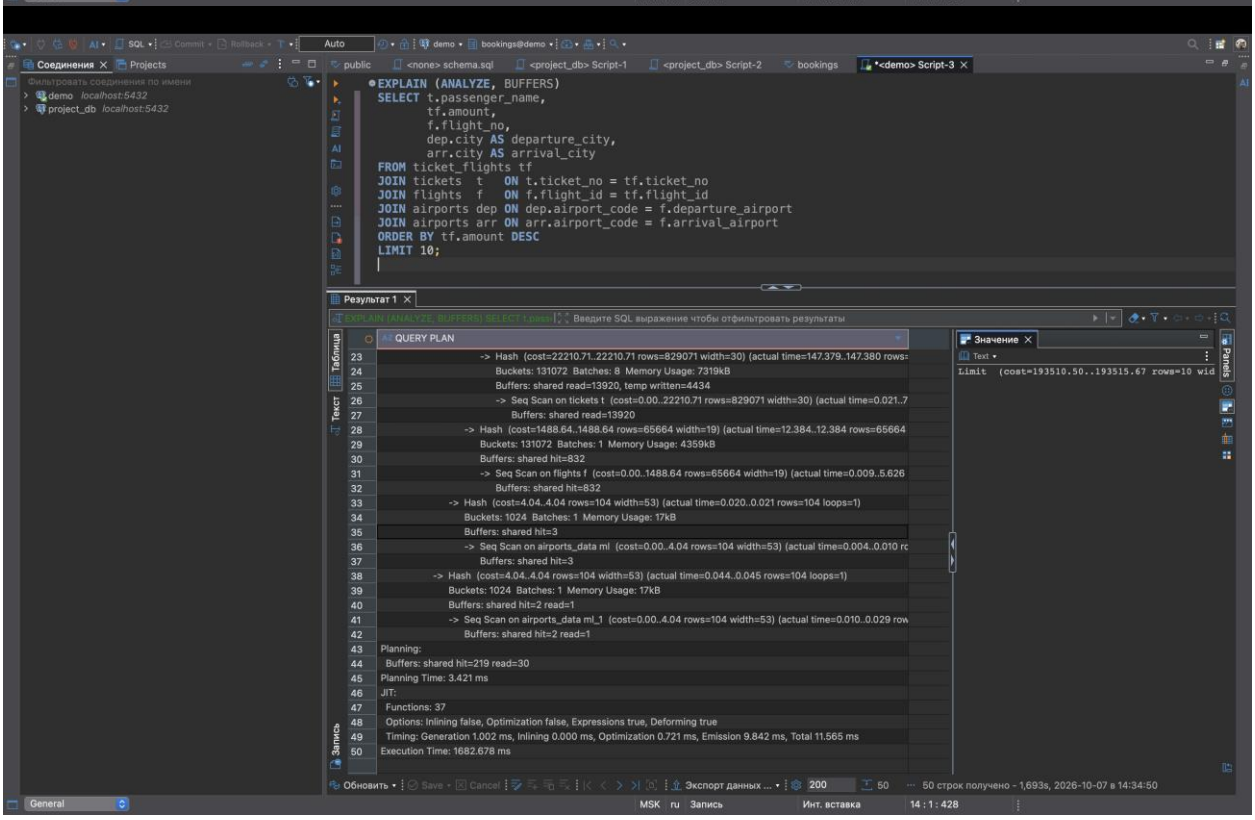
```
EXPLAIN (ANALYZE, BUFFERS)
SELECT t.passenger_name,
       tf.amount,
       f.flight_no,
       dep.city AS departure_city,
       arr.city AS arrival_city
FROM ticket_flights tf
JOIN tickets  t   ON t.ticket_no = tf.ticket_no
JOIN flights  f   ON f.flight_id = tf.flight_id
JOIN airports dep ON dep.airport_code = f.departure_airport
JOIN airports arr ON arr.airport_code = f.arrival_airport
ORDER BY tf.amount DESC
LIMIT 10;
```

ПРОСТОЙ ЗАПУСК

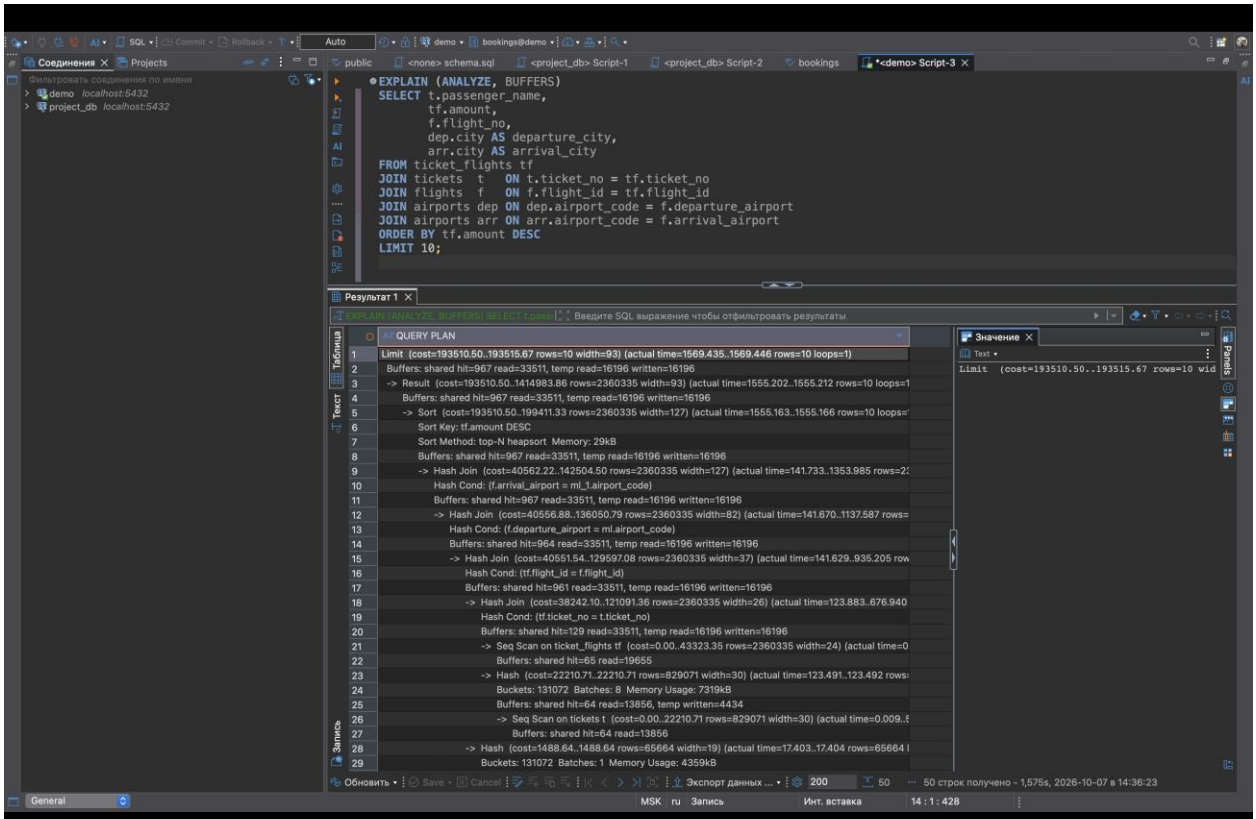


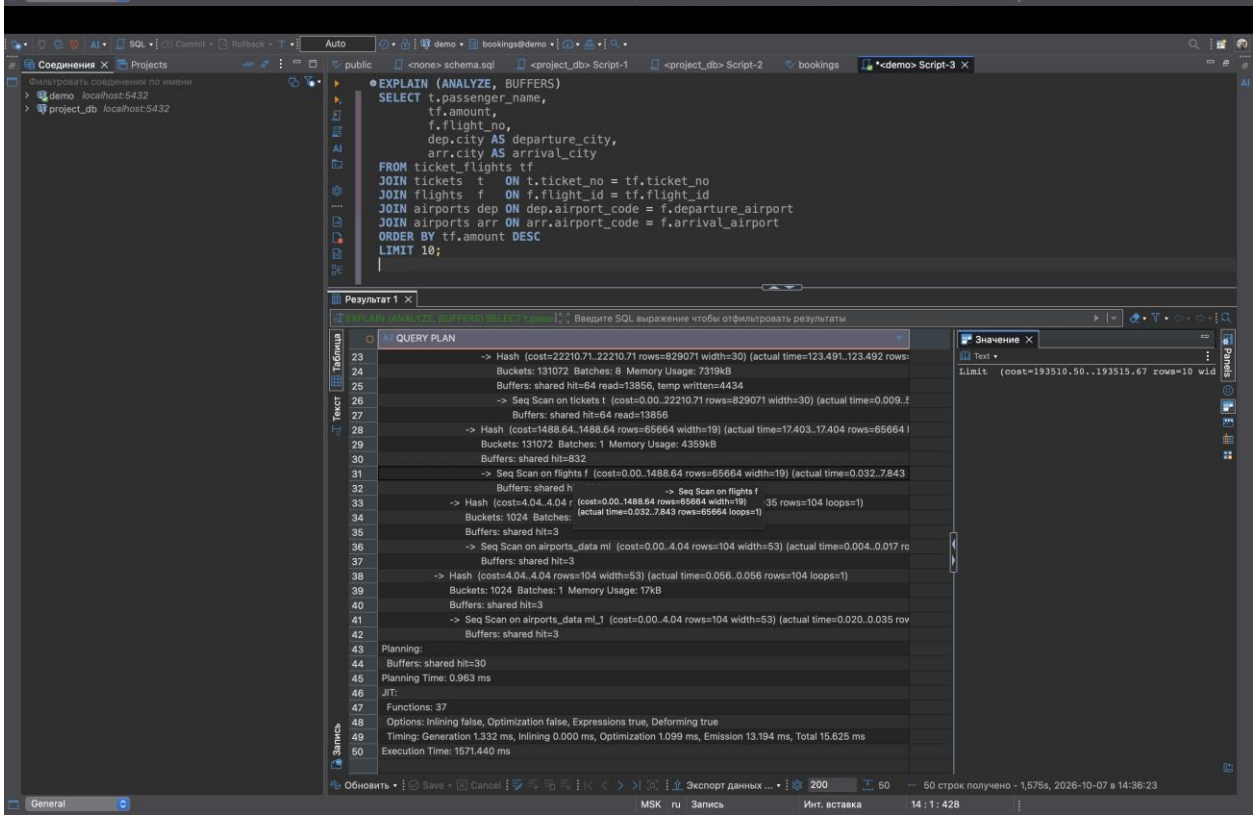
1 ЗАПУСК с EXPLAIN





2 ЗАПУСК с EXPLAIN





Пояснение:

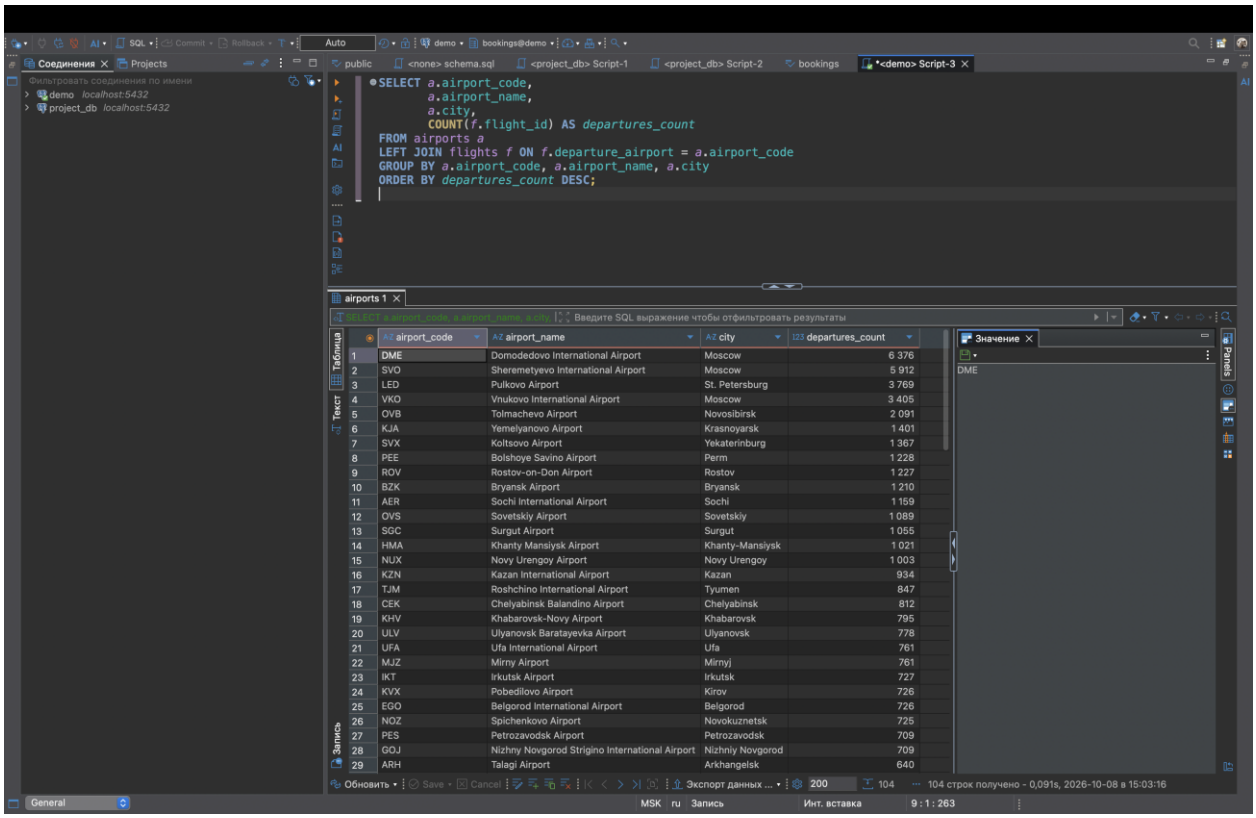
Запрос работал с диском при обоих запусках: при первом shared read=33 645 (ticket_flights 19 720 страниц, tickets 13 920), при втором shared read=33 511 при hit всего 967, то есть большие таблицы не помещаются в кеш и при повторном запуске читаются заново. В кеше оказались только flights (hit=832) и маленькая таблица аэропортов. Данные не поместились в память под хеш-таблицу: Hash по tickets разбит на 8 батчей и использует временные файлы (temp read=16 196, written=16 196). Индексы не использовались: все

таблицы читались полным Seq Scan (в ticket_flights 2 360 335 строк), соединения выполнены через Hash Join, а для LIMIT 10 применена сортировка top-N heapsort в памяти (29 kB). Время выполнения 1683 мс при первом запуске и 1571 мс при втором, плюс 11–16 мс на JIT-компиляцию.

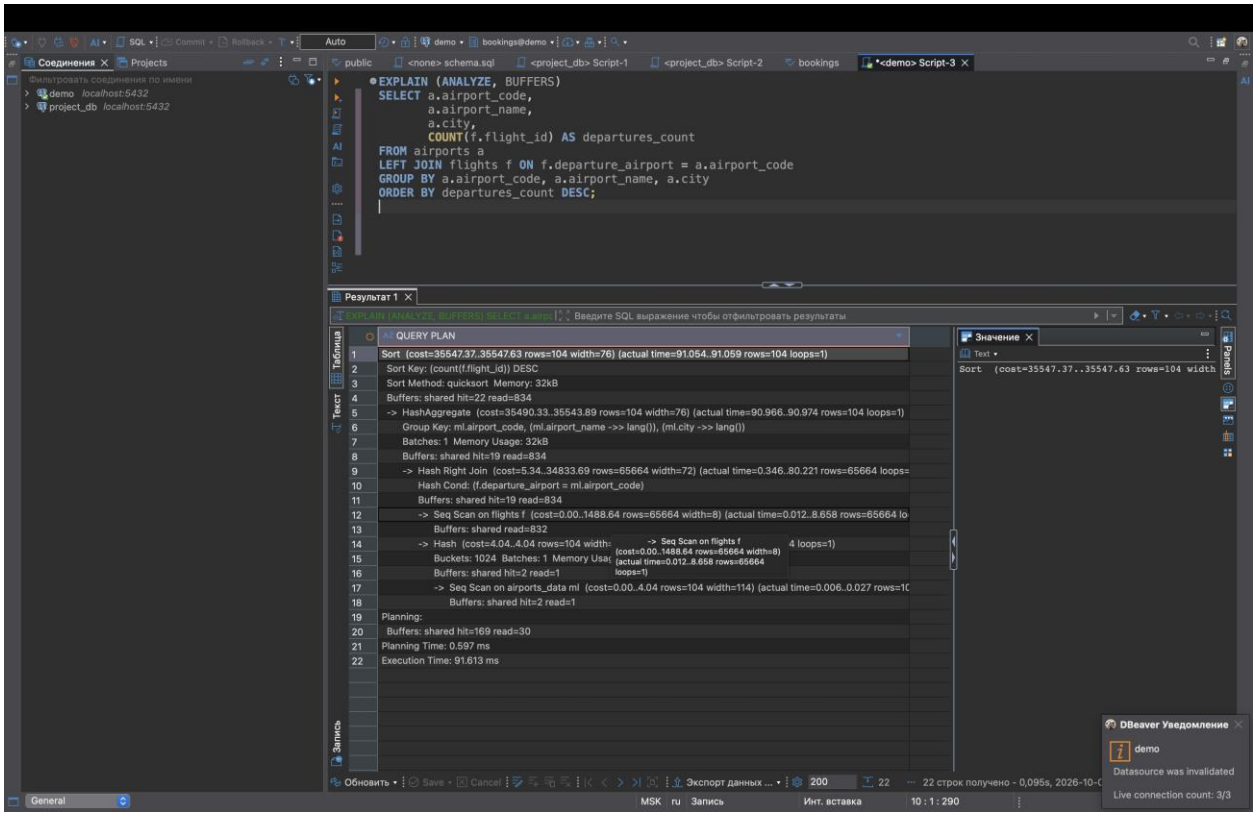
Задача 3.2. Все аэропорты с количеством вылетающих рейсов, включая нулевые:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT a.airport_code,
    a.airport_name,
    a.city,
    COUNT(f.flight_id) AS departures_count
FROM airports a
LEFT JOIN flights f ON f.departure_airport = a.airport_code
GROUP BY a.airport_code, a.airport_name, a.city
ORDER BY departures_count DESC;
```

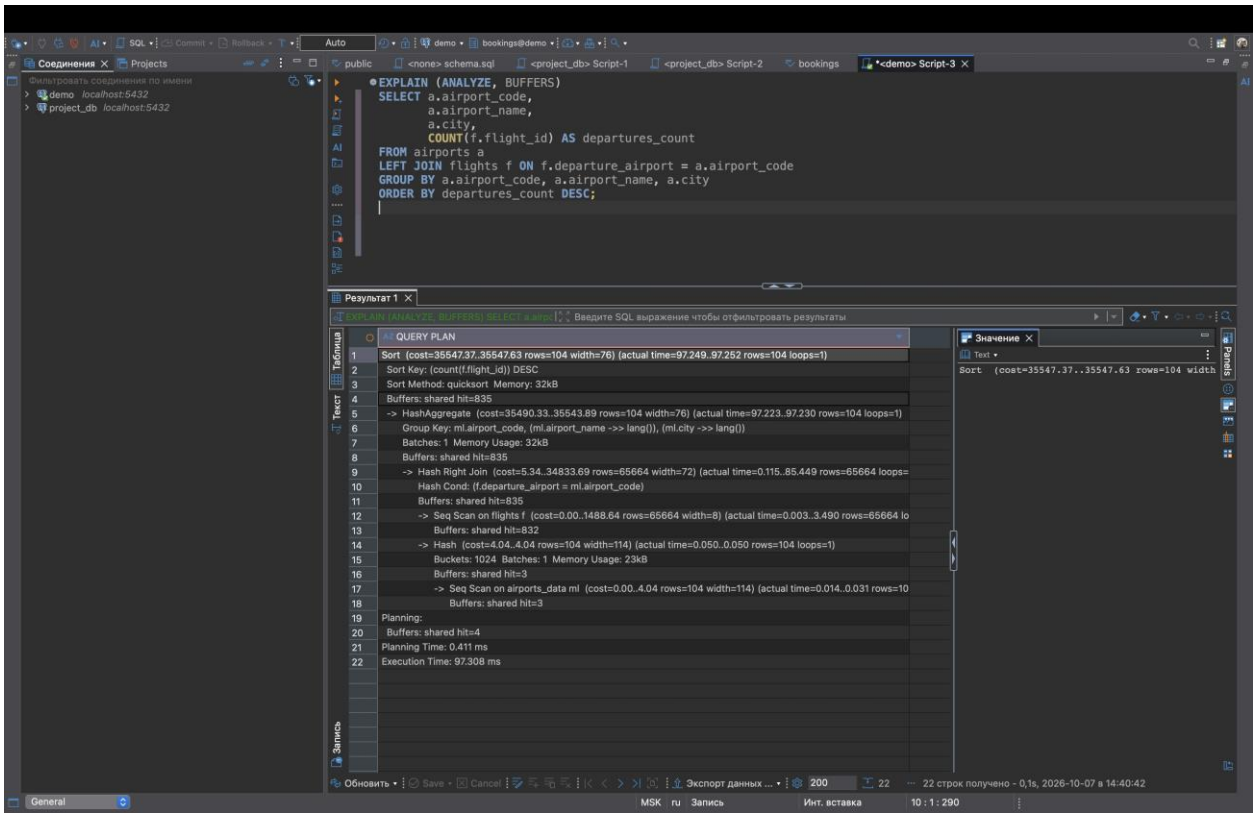
ПРОСТОЙ ЗАПУСК



1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



Пояснение:

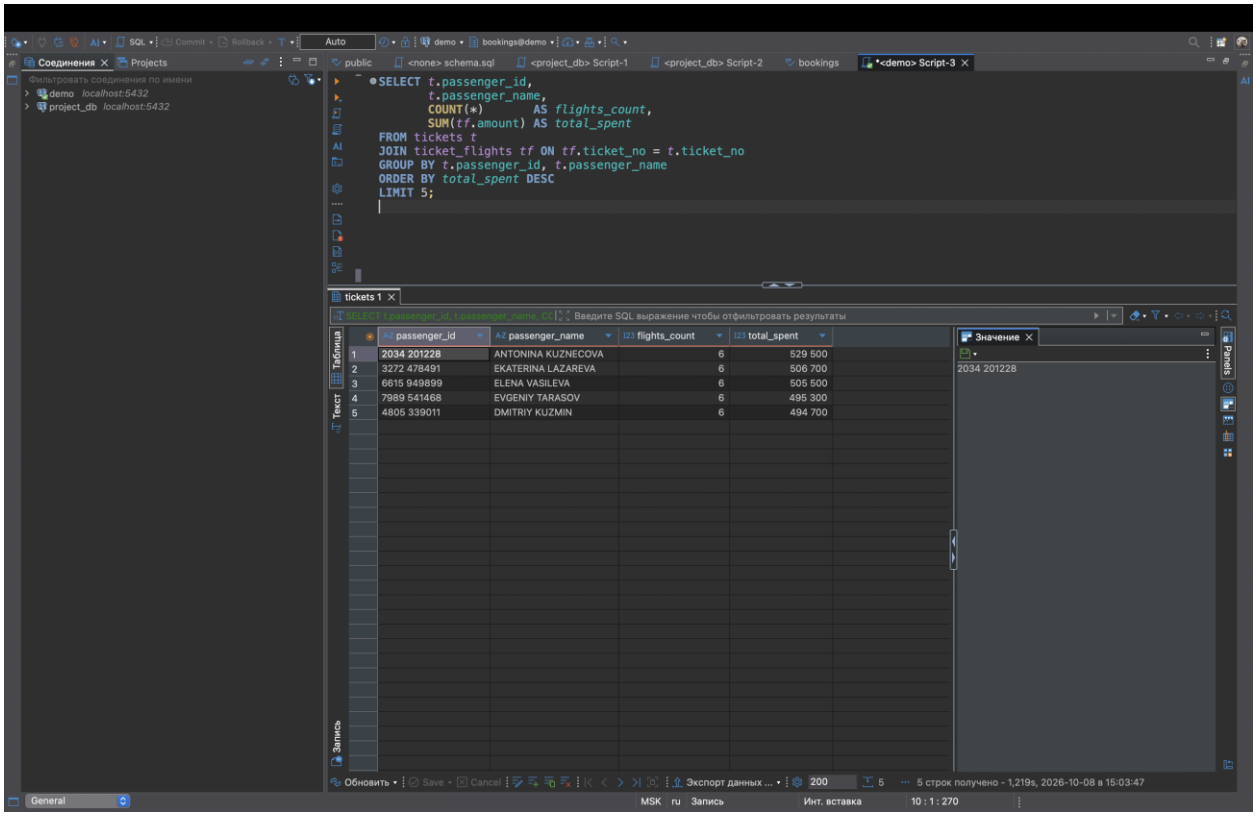
При первом запуске запрос работал с диском: таблица flights читалась с диска (shared read=832, всего read=834), а при втором запуске все страницы уже лежали в кеше (shared hit=835, read нет). Время планирования снизилось с 0.60 до 0.41 мс, а выполнение почти не изменилось (91.6 и 97.3 мс), потому что основное время уходит на соединение и агрегацию 65 664 строк, а не на чтение страниц. Индекс не использовался: flights (65 664

строки) и airports_data (104 аэропорта) читались полным Seq Scan, соединение выполнено через Hash Right Join, после чего HashAggregate сгруппировал строки по аэропортам (104 группы). Сортировка по количеству вылетов прошла в памяти (quicksort, 32 kB), временные файлы не использовались.

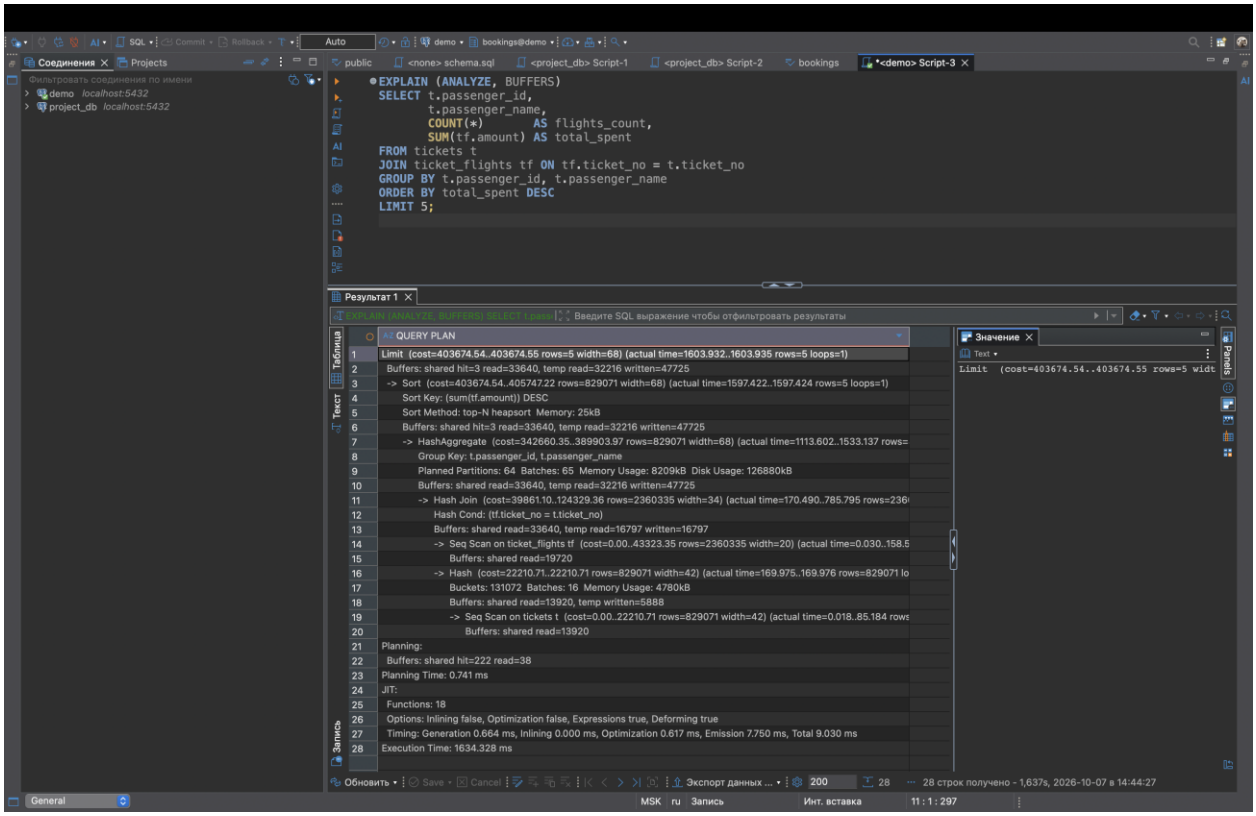
Задача 3.3. Топ-5 пассажиров по суммарным тратам:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT t.passenger_id,
       t.passenger_name,
       COUNT(*)        AS flights_count,
       SUM(tf.amount) AS total_spent
FROM tickets t
JOIN ticket_flights tf ON tf.ticket_no = t.ticket_no
GROUP BY t.passenger_id, t.passenger_name
ORDER BY total_spent DESC
LIMIT 5;
```

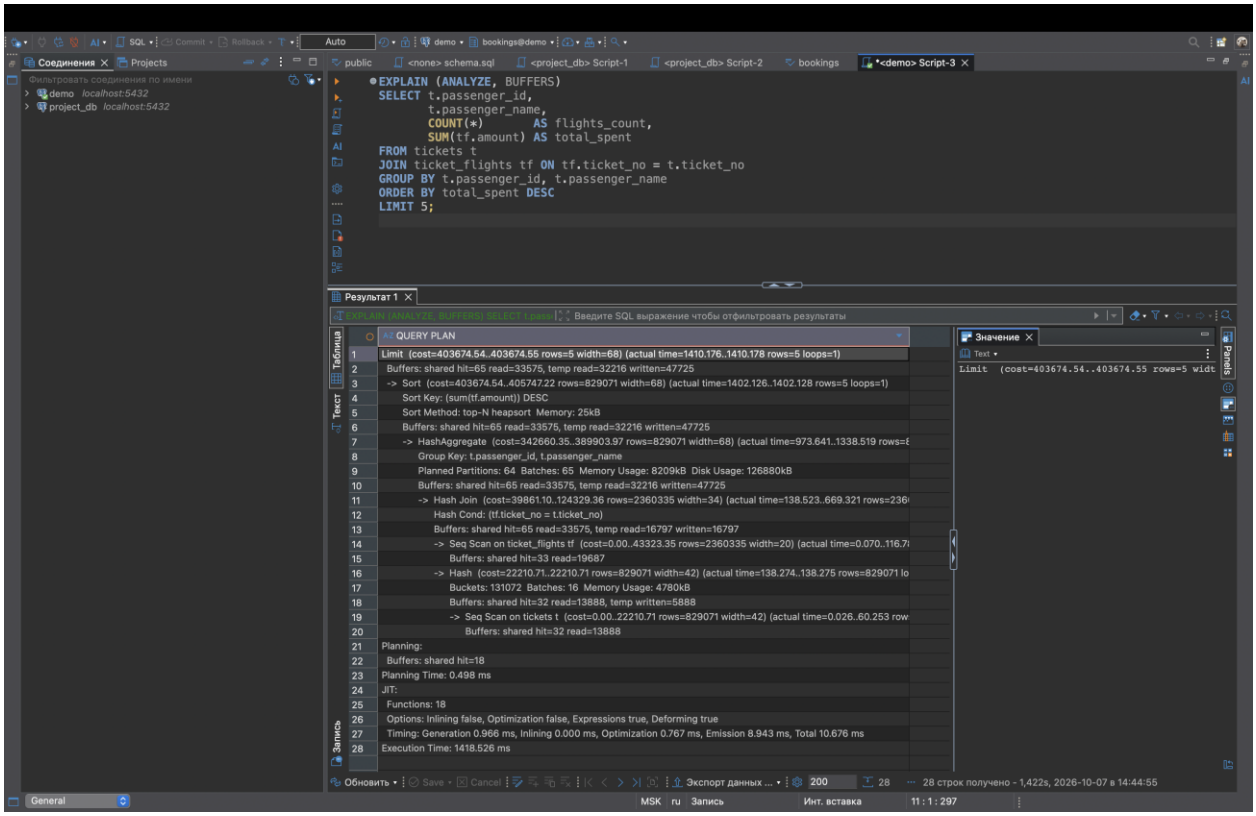
ПРОСТОЙ ЗАПУСК



1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



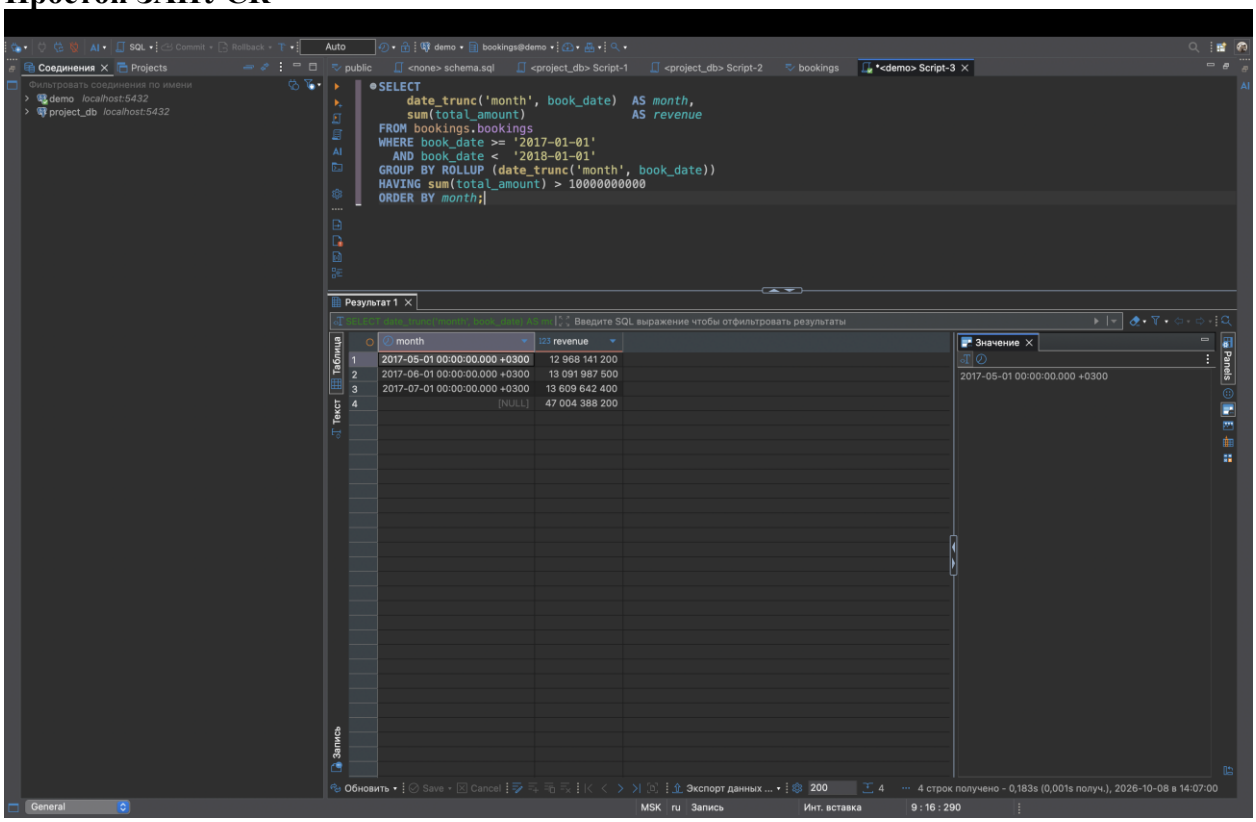
Пояснение:

Запрос работал с диском при обоих запусках: при первом shared read=33 640 (hit всего 3), при втором shared read=33 575 (hit=65), то есть таблицы ticket_flights (около 19 700 страниц) и tickets (около 13 900) почти целиком читались заново, и повторный запуск кеш не ускорил. Кроме того, запрос использовал временные файлы на диске: HashAggregate

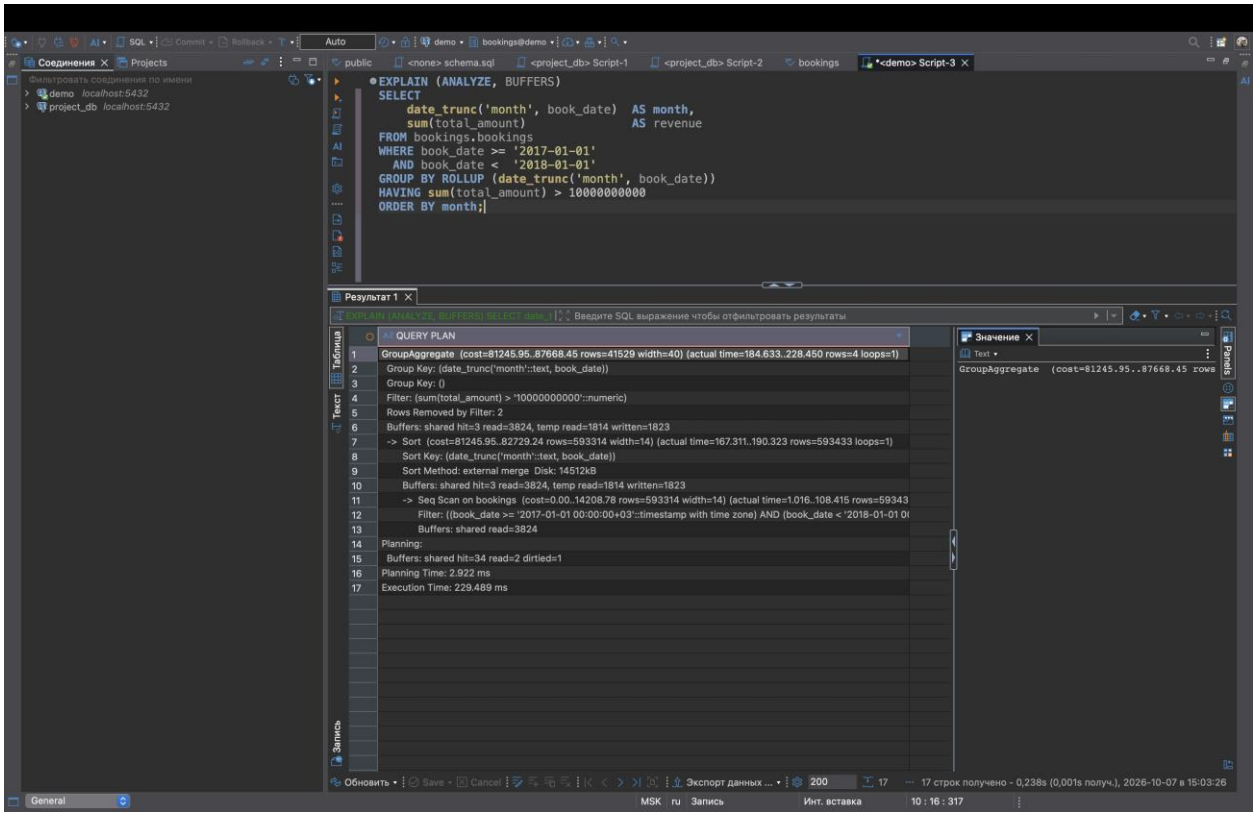
разбит на 65 партий (Disk Usage: 126 880 kB), Hash по tickets шёл в 16 батчей (temp read=32 216, written=47 725). Индексы не использовались: обе таблицы читались полным Seq Scan (в ticket_flights 2 360 335 строк), соединение выполнено через Hash Join, а для LIMIT 5 применена сортировка top-N heapsort в памяти (25 kB). Время выполнения 1634 мс при первом запуске и 1419 мс при втором, плюс около 9–11 мс на JIT-компиляцию.

1.Группировки и многоуровневые итоги (ROLLUP): Посчитайте помесячную выручку от продажи билетов за 2017 год вместе с общим итогом — одним запросом, через ROLLUP.

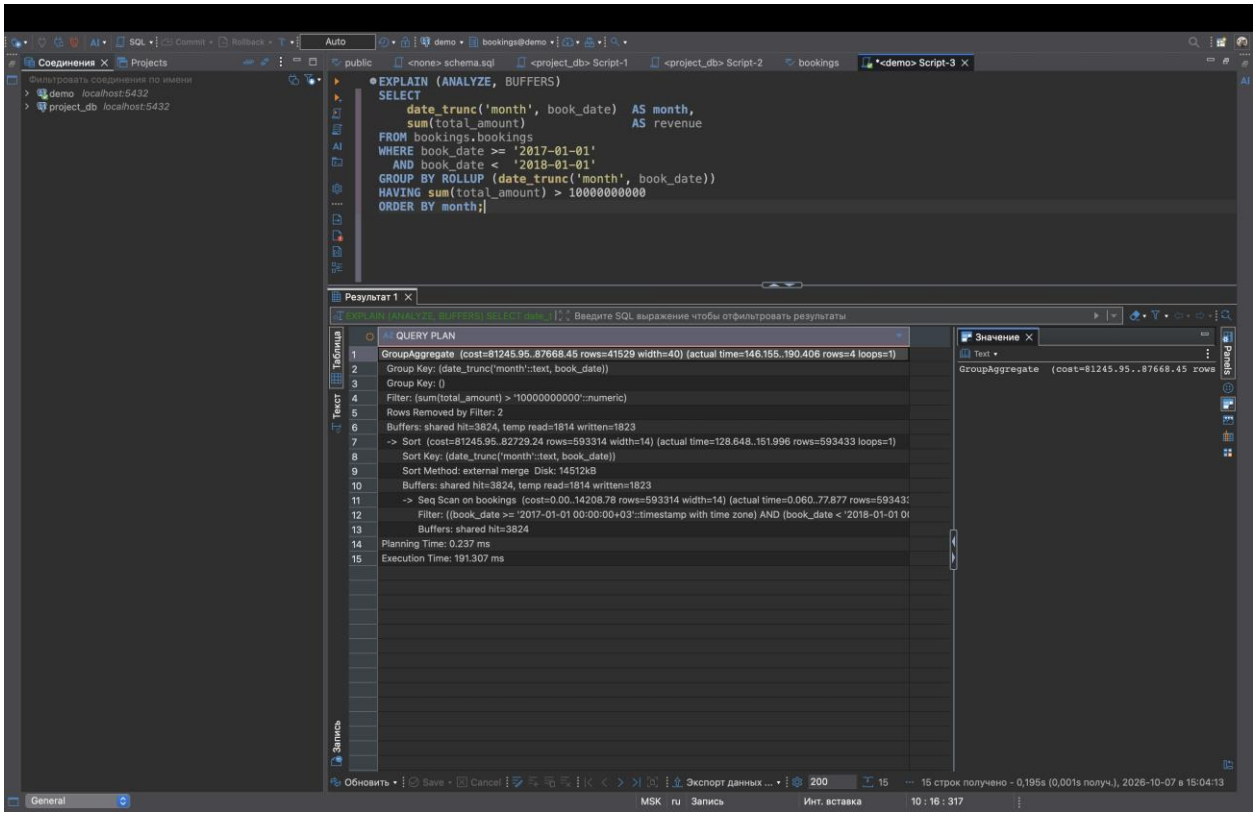
Простой ЗАПУСК



1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



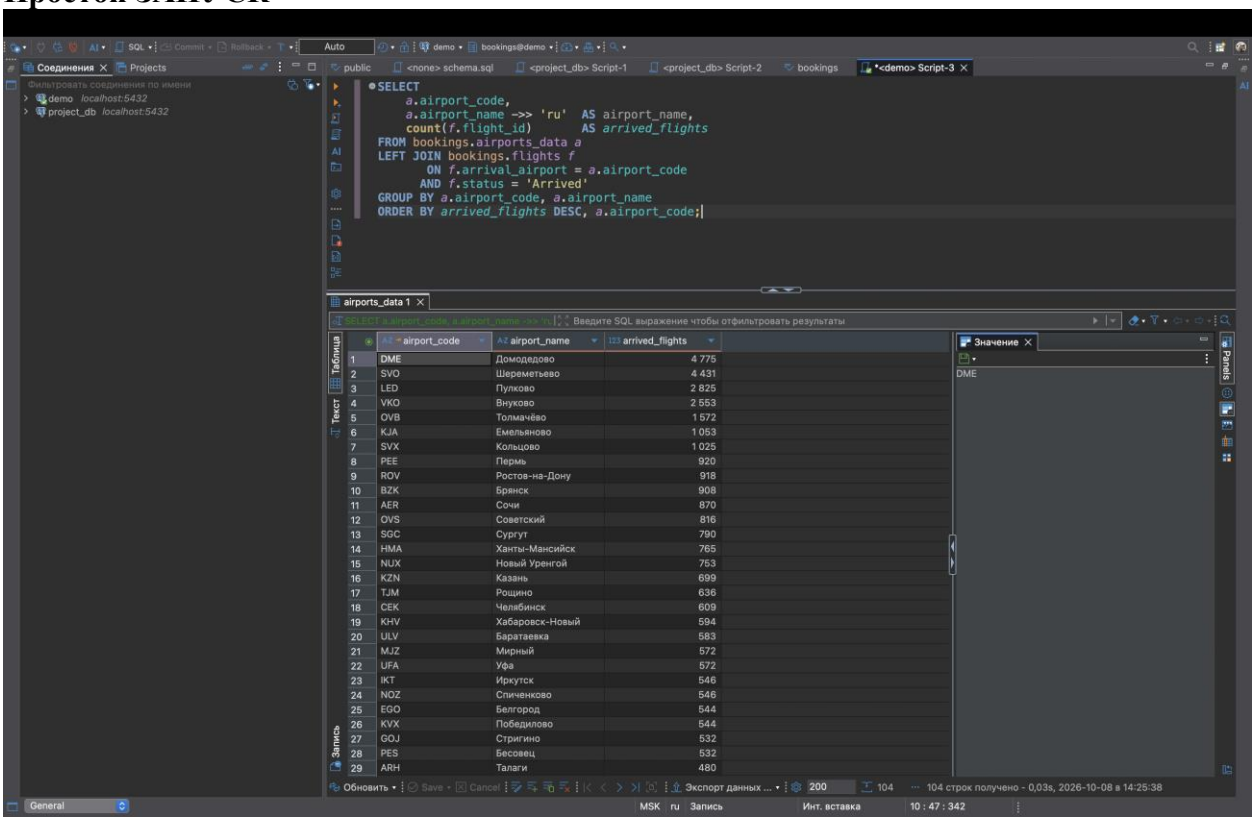
ПОЯСНЕНИЕ:

При первом запуске запрос работал с диском: bookings читалась с диска (shared read=3824), при втором все 3824 страницы были в кеше (shared hit=3824, read нет). Время выполнения снизилось с 229,5 до 191,3 мс, планирования с 2,92 до 0,24 мс. Индекс не использовался: Seq Scan по bookings с фильтром по book_date (читается почти вся

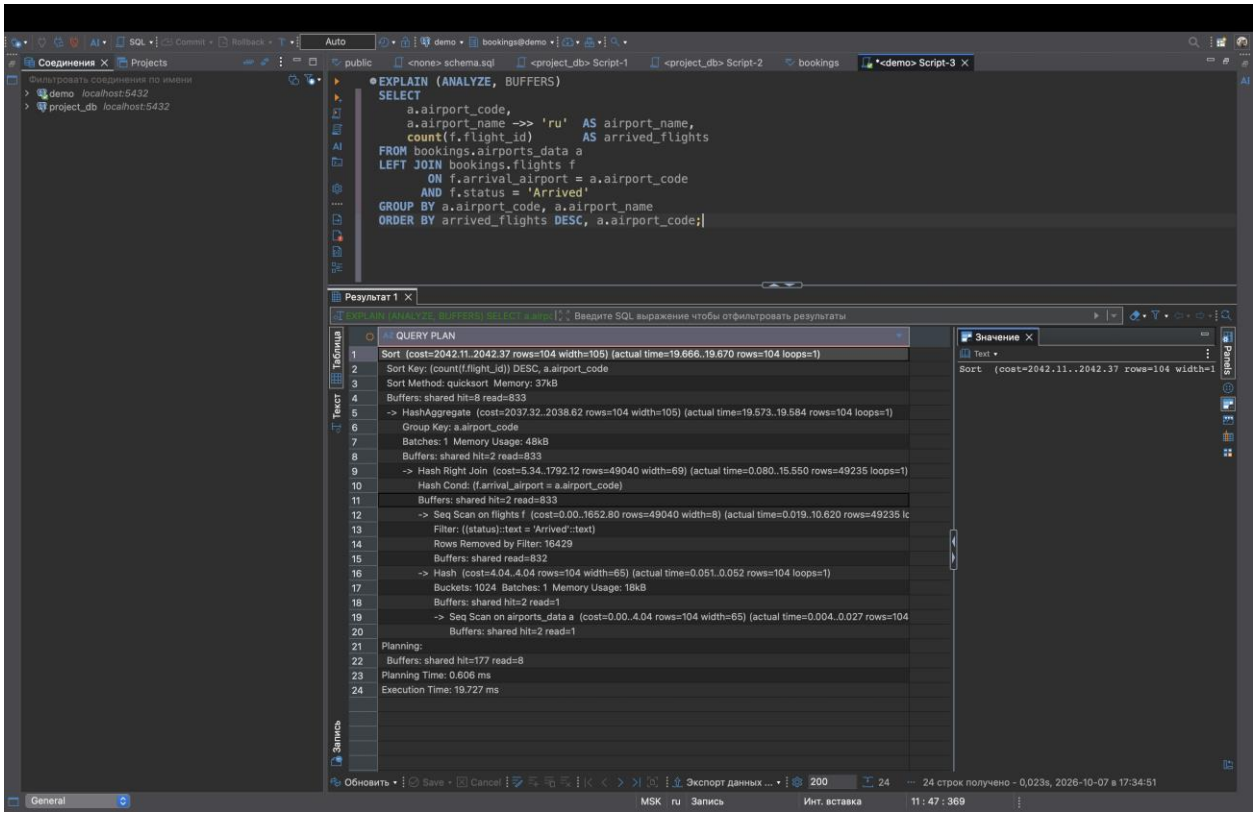
таблица, около 593 тыс. строк), поэтому индекс здесь всё равно не дал бы выигрыша. Сортировка для GroupAggregate не уместилась в память: external merge, Disk: 14512kB, и временные файлы использовались при обоих запусках (temp read=1814 written=1823), кеш этого не убрал.
GROUP BY ROLLUP строит две группировки, что видно в плане по двум строкам Group Key: по месяцу и пустая Group Key: (), то есть общий итог. HAVING применяется как Filter: (sum(total_amount) > '10000000000') в узле GroupAggregate уже после агрегации, к каждой группе отдельно: из 6 групп отсечено 2 (Rows Removed by Filter: 2), осталось 4 (rows=4). Итоговая строка с NULL в колонке месяца проходит фильтр, потому что условие смотрит только на sum(total_amount), а не на сам месяц. Сумма за весь год заведомо больше порога в 10 млрд, а NULL здесь лишь маркер группы, который не участвует в сравнении.

2.LEFT JOIN и условная агрегация: Для каждого аэропорта прибытия посчитайте число прибывших рейсов (status = 'Arrived'), включая аэропорты, куда не прибыл ни один рейс: код аэропорта, название, число рейсов.

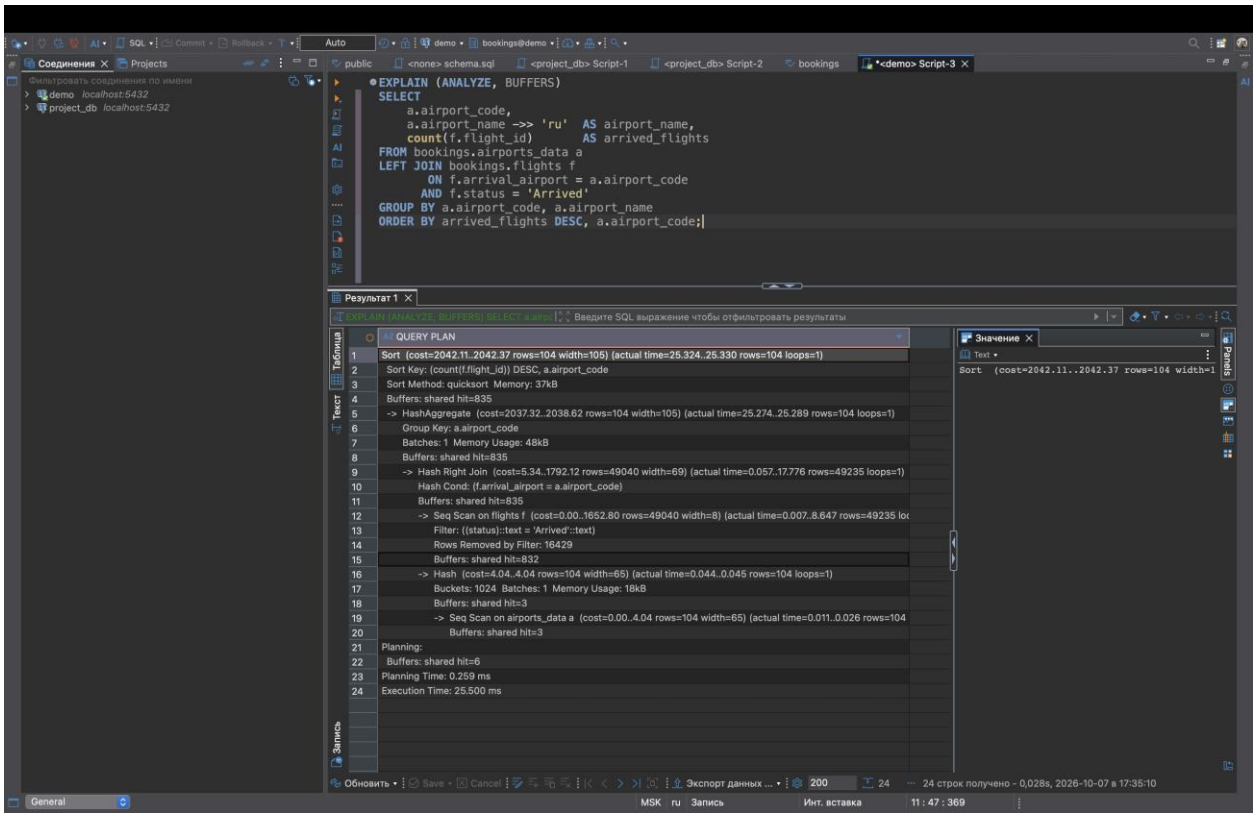
Простой ЗАПУСК



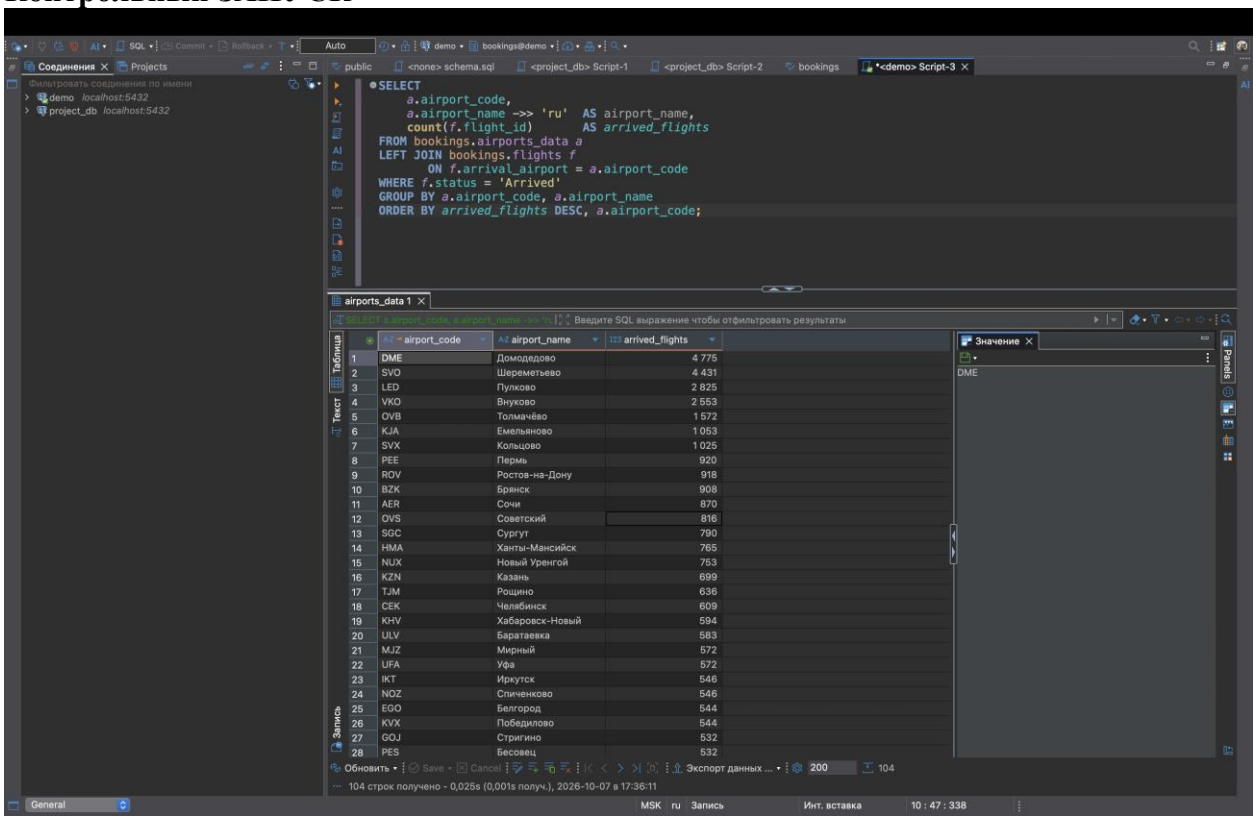
1 ЗАПУСК с EXPLAIN



2 ЗАПУСК с EXPLAIN



Контрольный ЗАПУСК



ПОЯСНЕНИЕ:

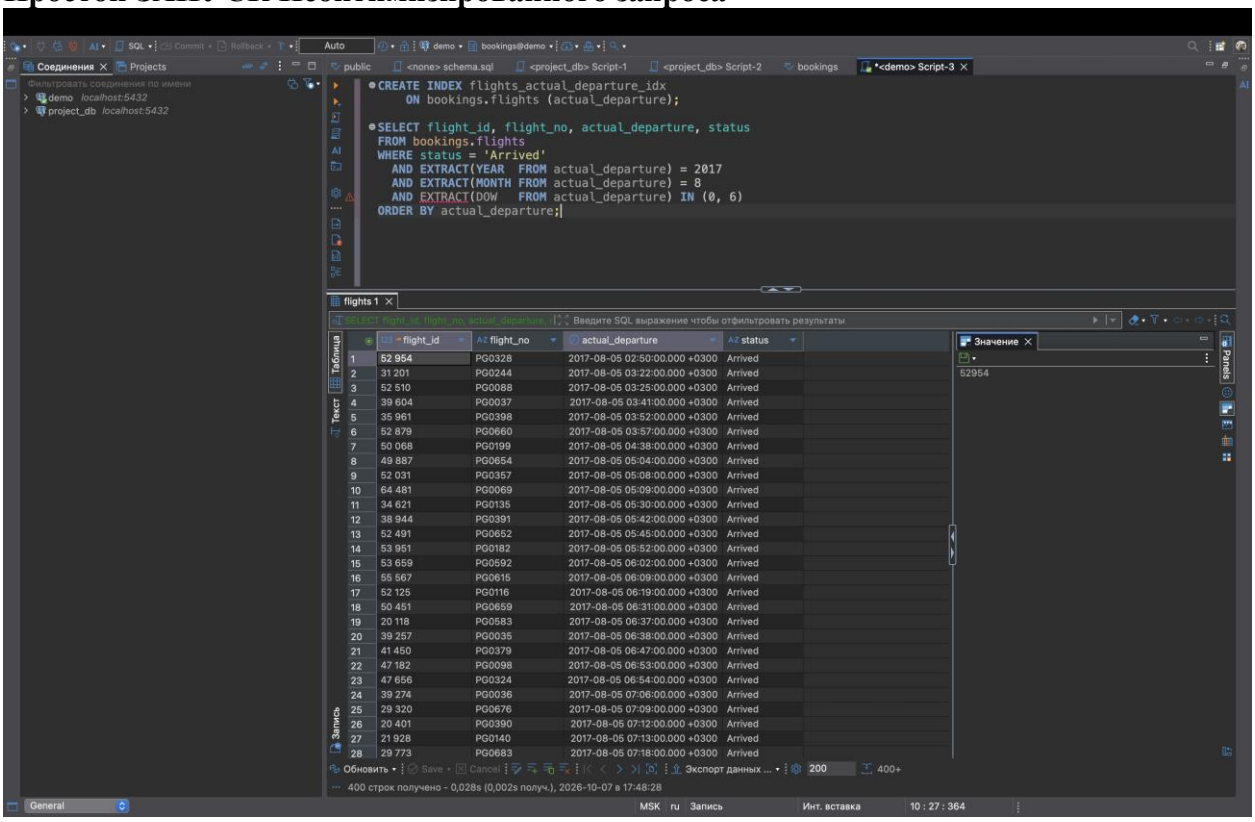
При первом запуске запрос работал с диском: flights читалась с диска (shared read=832, всего read=833, hit=8), при втором все страницы были в кеше (shared hit=835, read нет). Планирование снизилось с 0,61 до 0,26 мс, а выполнение почти не изменилось (19,7 и 25,6 мс), так как запрос короткий и разница в пределах шума. Индекс не использовался: Seq Scan по flights с фильтром status = 'Arrived' (получено 49 235 строк, отброшено 16 429) и по airports_data (104 аэропорта). Хеш строится по airports_data, затем идут HashAggregate по 104 группам и сортировка в памяти (quicksort, 37 kB), временных файлов нет.

COUNT(*) здесь недопустим: для аэропорта без подходящих рейсов LEFT JOIN возвращает одну строку с NULL в колонках flights, и COUNT(*) насчитает 1 вместо 0. COUNT(f.flight_id) не учитывает NULL, поэтому даёт верный 0. В плане Hash Right Join означает, что планировщик построил хеш-таблицу по маленькой airports_data и прогнал через неё flights. Аэропорты без совпадений дополняются NULL после прохода, так что логика результата остаётся как у LEFT JOIN.

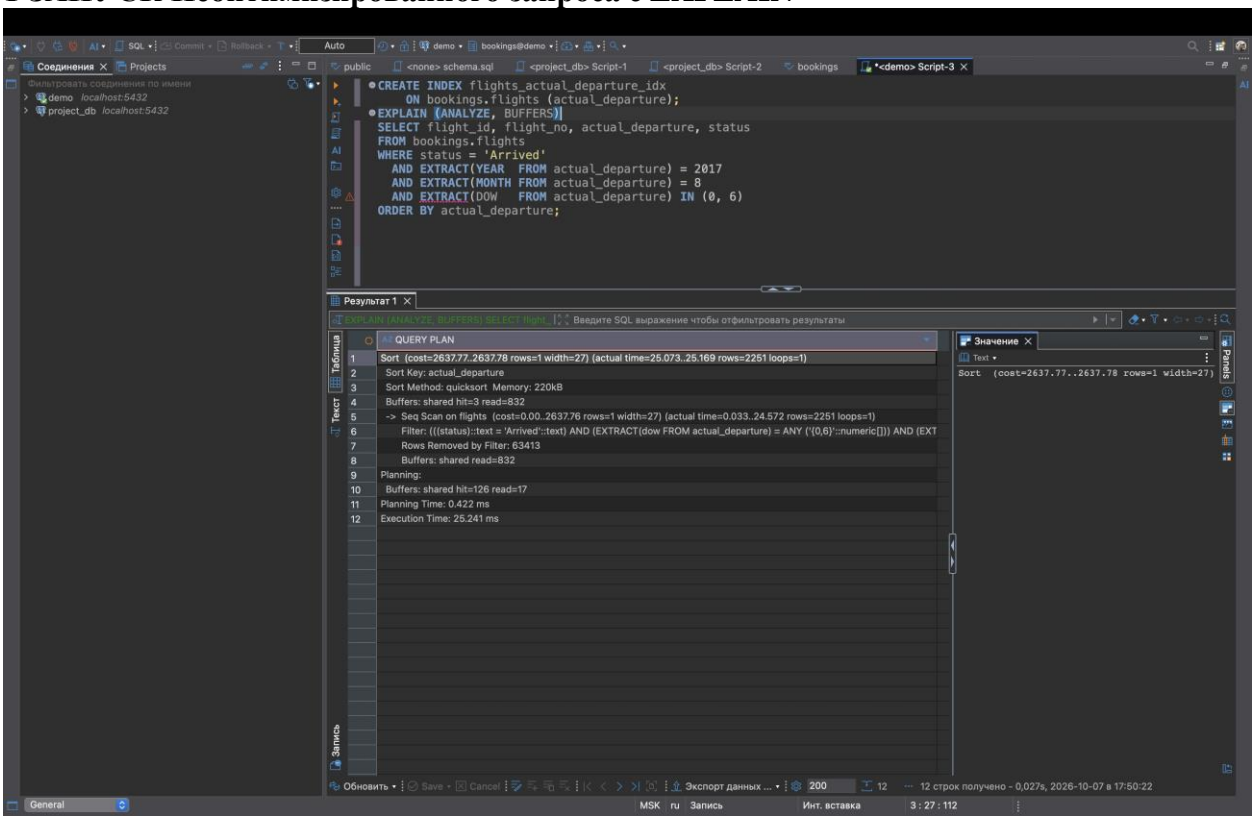
При переносе условия f.status = 'Arrived' из ON в WHERE результат получился такой же: 104 строки и те же счётчики. Это связано с данными: в базе demo в каждый из 104 аэропортов прибыл хотя бы один рейс со статусом Arrived, поэтому аэропортов с нулём нет и WHERE не отбросил ни одной строки. В общем случае такой перенос опасен: для аэропорта без подходящих рейсов LEFT JOIN формирует строку с NULL в f.status, условие NULL = 'Arrived' не выполняется, и строка отбрасывается, то есть LEFT JOIN превращается в INNER JOIN, а аэропорты с нулём пропадают. На этих данных разница не проявилась, но корректной остаётся только форма с условием в ON.

3.Sargability (Оптимизация предикатов): Найдите все рейсы, выполненные (status = 'Arrived') в выходные дни (суббота и воскресенье) в августе 2017 года.

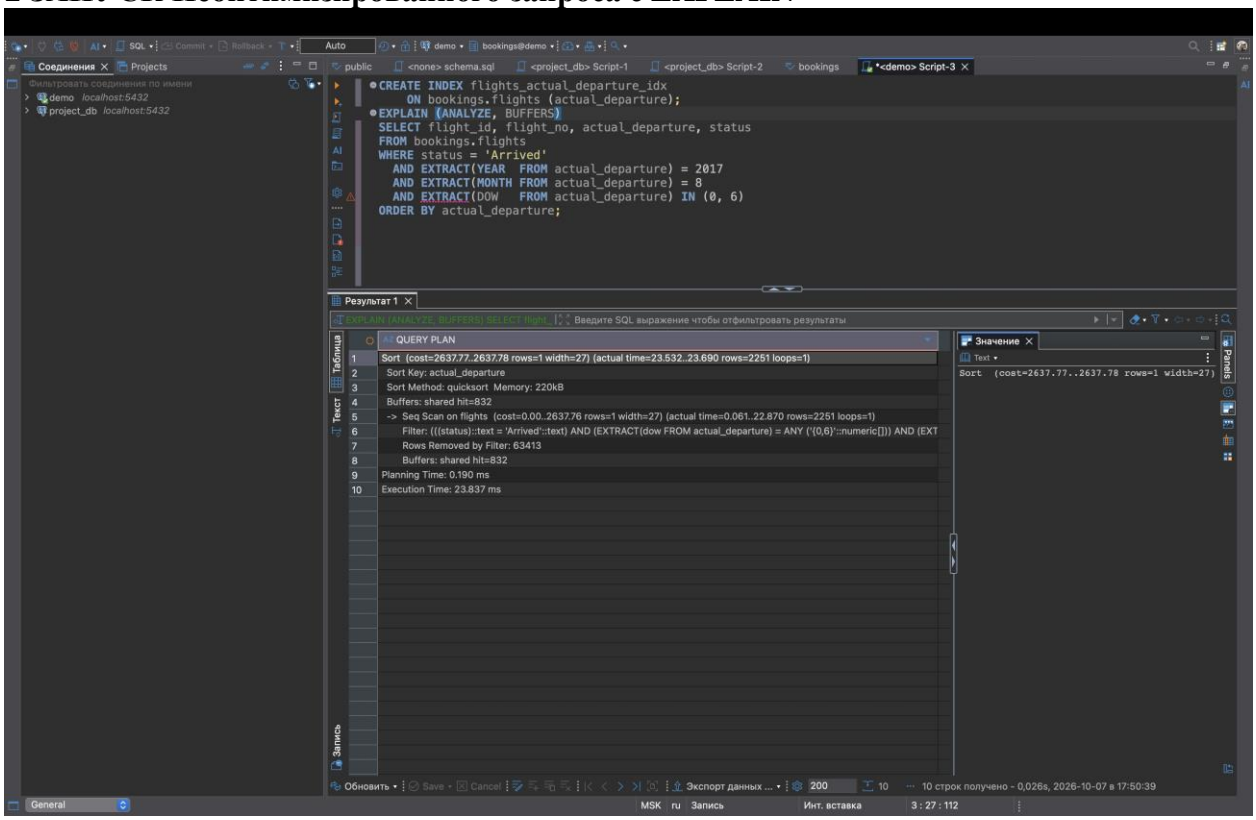
Простой ЗАПУСК Неоптимизированного запроса



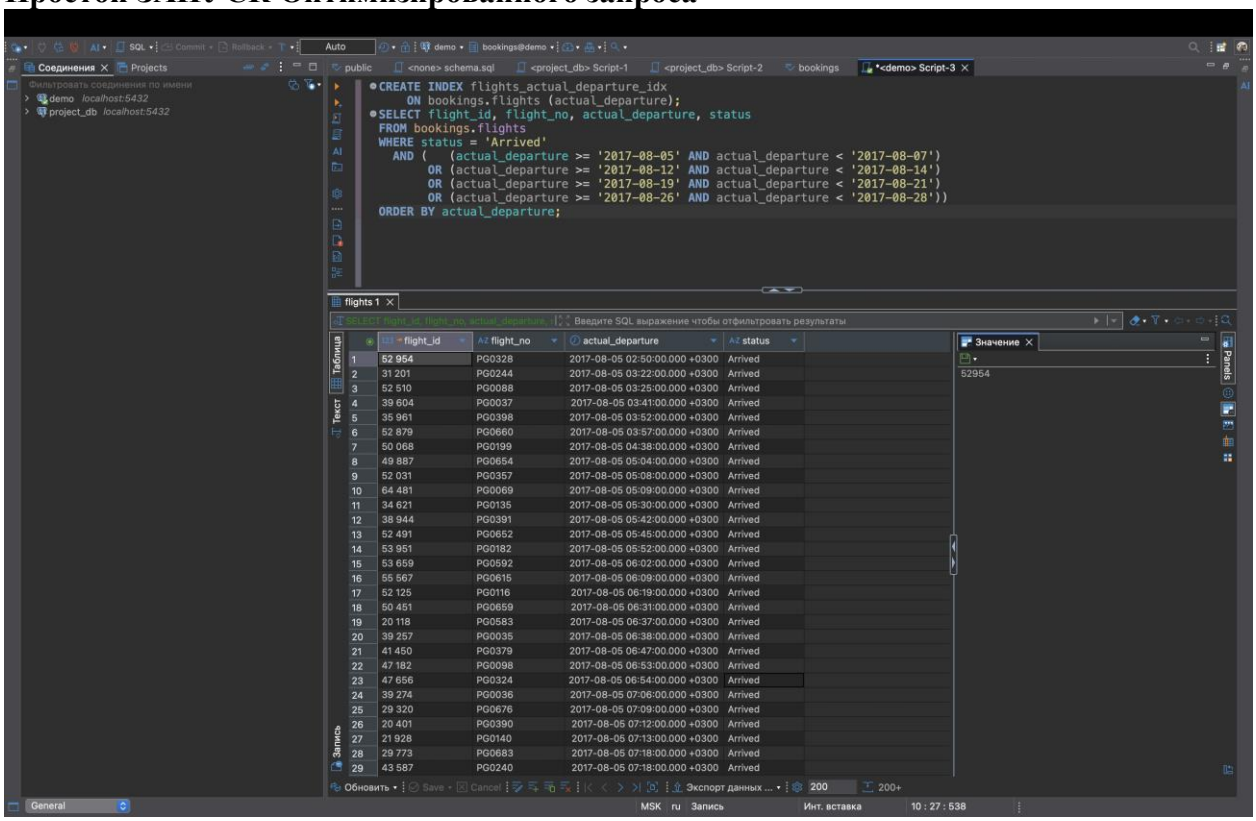
1 ЗАПУСК Неоптимизированного запроса с EXPLAIN



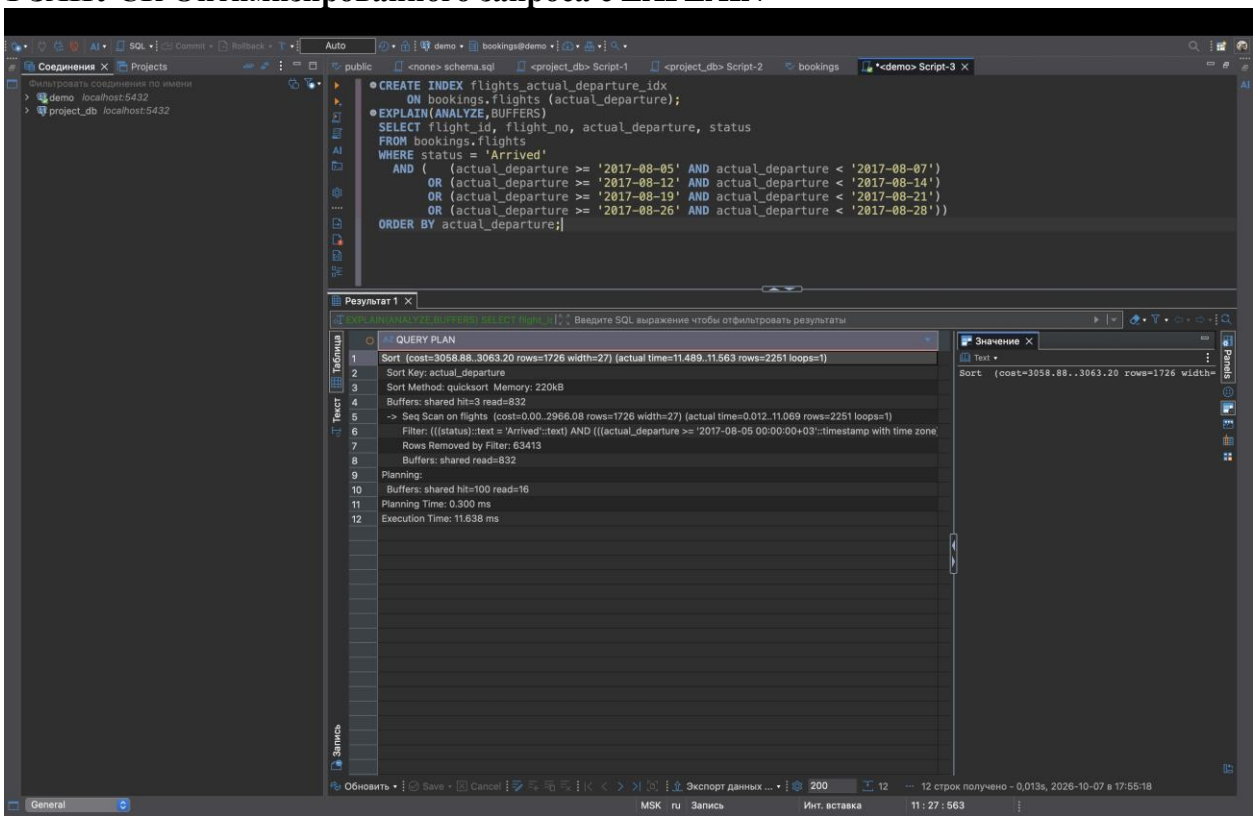
2 ЗАПУСК Неоптимизированного запроса с EXPLAIN



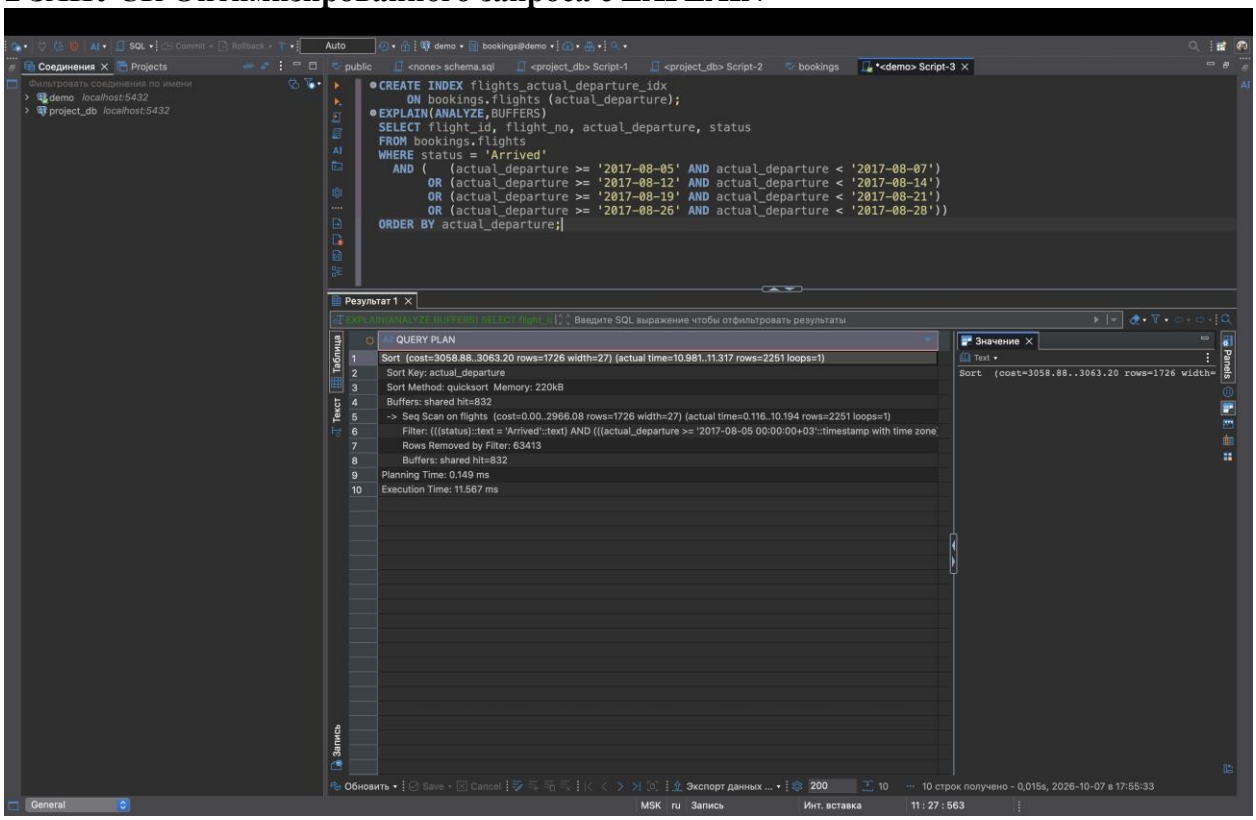
Простой ЗАПУСК Оптимизированного запроса



1 ЗАПУСК Оптимизированного запроса с EXPLAIN



2 ЗАПУСК Оптимизированного запроса с EXPLAIN



ПОЯСНЕНИЕ (неоптимизированный запрос, EXTRACT):

При первом запуске (17:50:22) запрос работал с диском: flights читалась целиком (shared read=832), при втором (17:50:39) все 832 страницы были в кеше (shared hit=832, read нет). Планирование снизилось с 0,42 до 0,19 мс, а выполнение изменилось слабо (25,2 и 23,8 мс). Индекс не использовался: функции EXTRACT(YEAR/MONTH/DOW FROM actual_departure) применяются к колонке, и поиск по индексу невозможен, поэтому Seq Scan проверяет все 65 664 строки (отброшено 63 413, получено 2251). Планировщик не может оценить такие условия, поэтому ожидал 1 строку (rows=1) вместо реальных 2251. Сортировка по actual_departure идёт в памяти (quicksort, 220 kB), временных файлов нет.

ПОЯСНЕНИЕ (оптимизированный запрос, диапазоны дат):

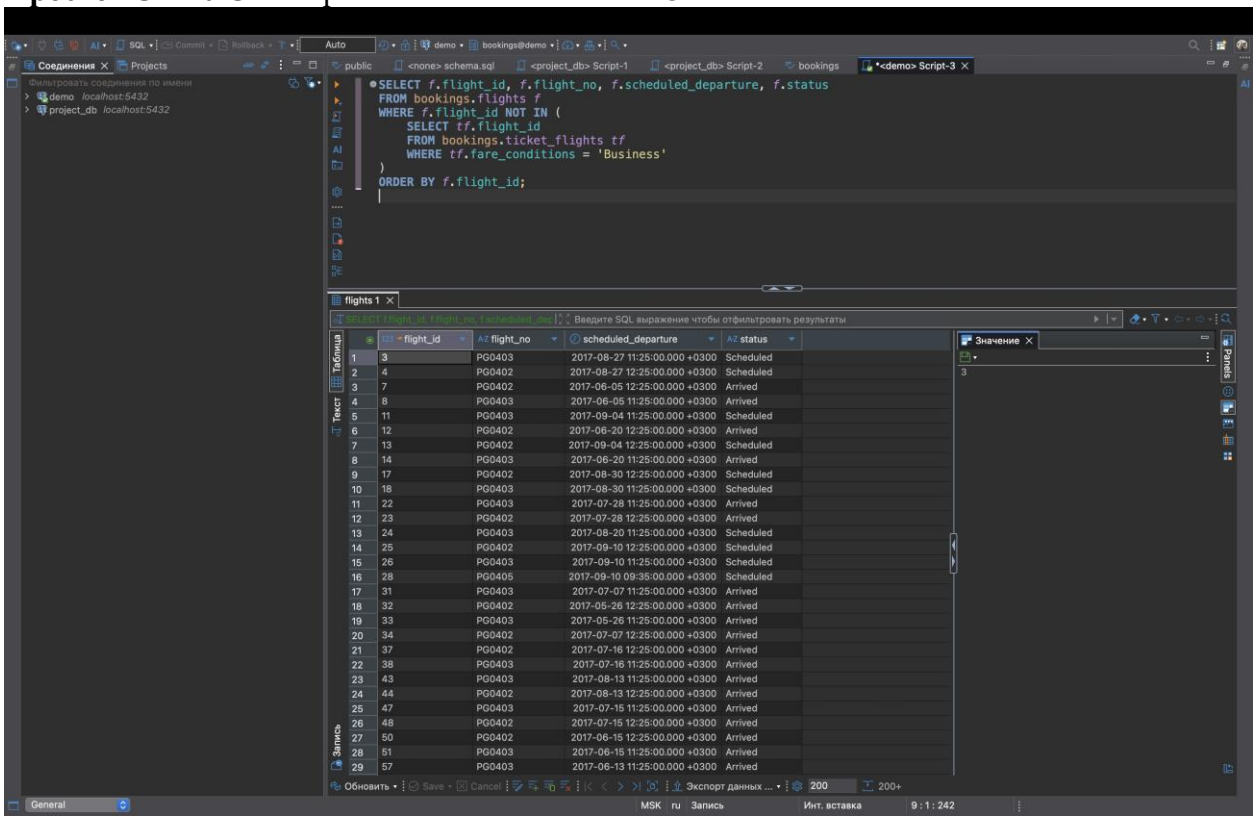
При первом запуске (17:55:18) запрос читал таблицу с диска (shared read=832), при втором (17:55:33) всё было в кеше (shared hit=832). Планирование снизилось с 0,30 до 0,15 мс, выполнение почти не изменилось (11,6 и 11,6 мс). План остался Seq Scan (отброшено 63 413, получено 2251), но условия actual_departure >= ... AND actual_departure < ... сравнивают колонку с константой, то есть sargable. Планировщик теперь оценивает выборку по статистике (rows=1726 против реальных 2251), а не вслепую. Строки проверяются простым сравнением, а не вычислением трёх функций EXTRACT для каждой строки.

Разница в планах (общий вывод):

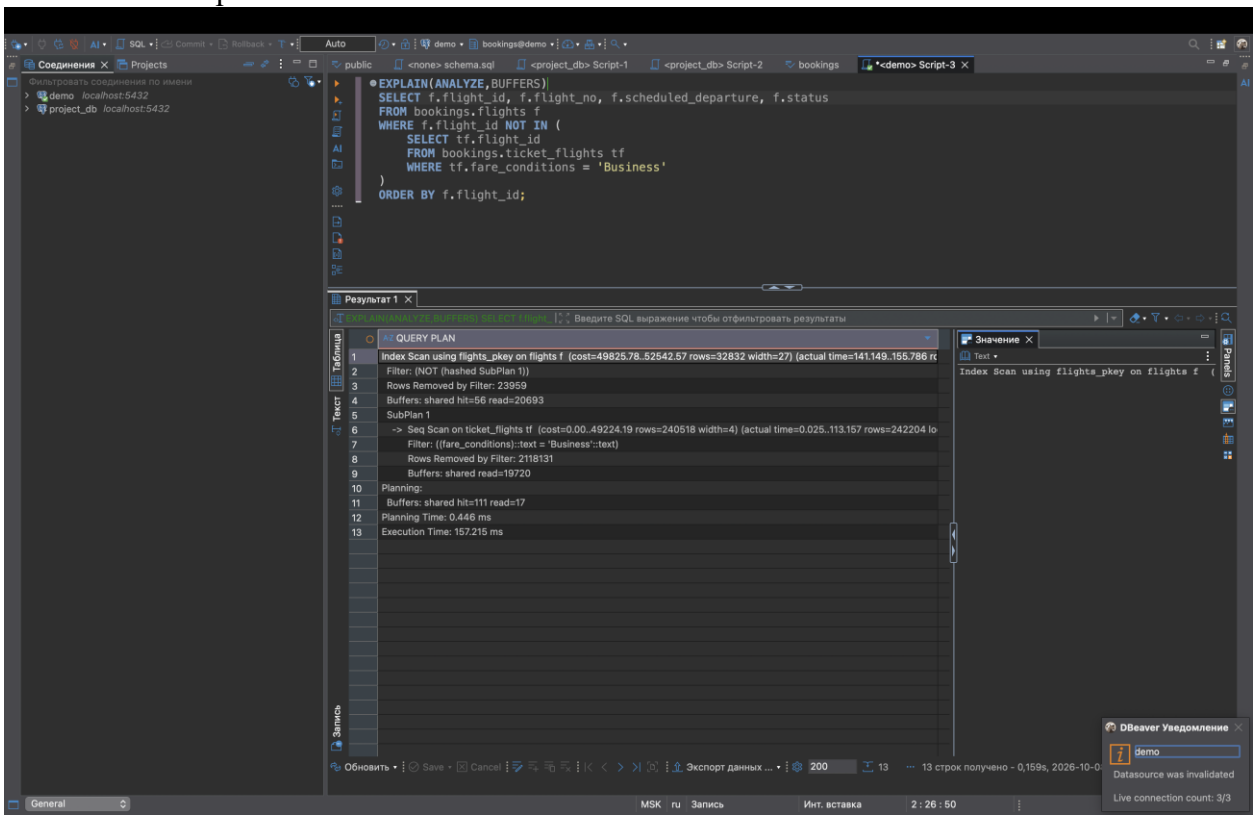
Результат у обоих запросов одинаковый (2251 строка), и оба прочитали одни и те же 832 страницы. Разница в стоимости проверки условия: оптимизированный запрос выполняется примерно вдвое быстрее (около 11,6 против 23,8-25,2 мс), потому что не вычисляет EXTRACT для каждой из 65 664 строк. Оценка числа строк тоже стала точнее (1726 против 1). Индекс при этом не сработал в обоих случаях: таблица маленькая (832 страницы), условие status = 'Arrived' с четырьмя диапазонами выбирает около 3,4% строк, и планировщик счёл полное чтение дешевле. Свойство sargable делает использование индекса возможным, но не гарантирует, что планировщик его выберет. На большой таблице оптимизированный запрос получил бы Index Scan или Bitmap Index Scan, а неоптимизированный остался бы на Seq Scan.

4.ANTI-JOIN (Оптимизация отрицания): Найдите все рейсы, на которые не был продан ни один билет бизнес-класса (fare_conditions = 'Business').

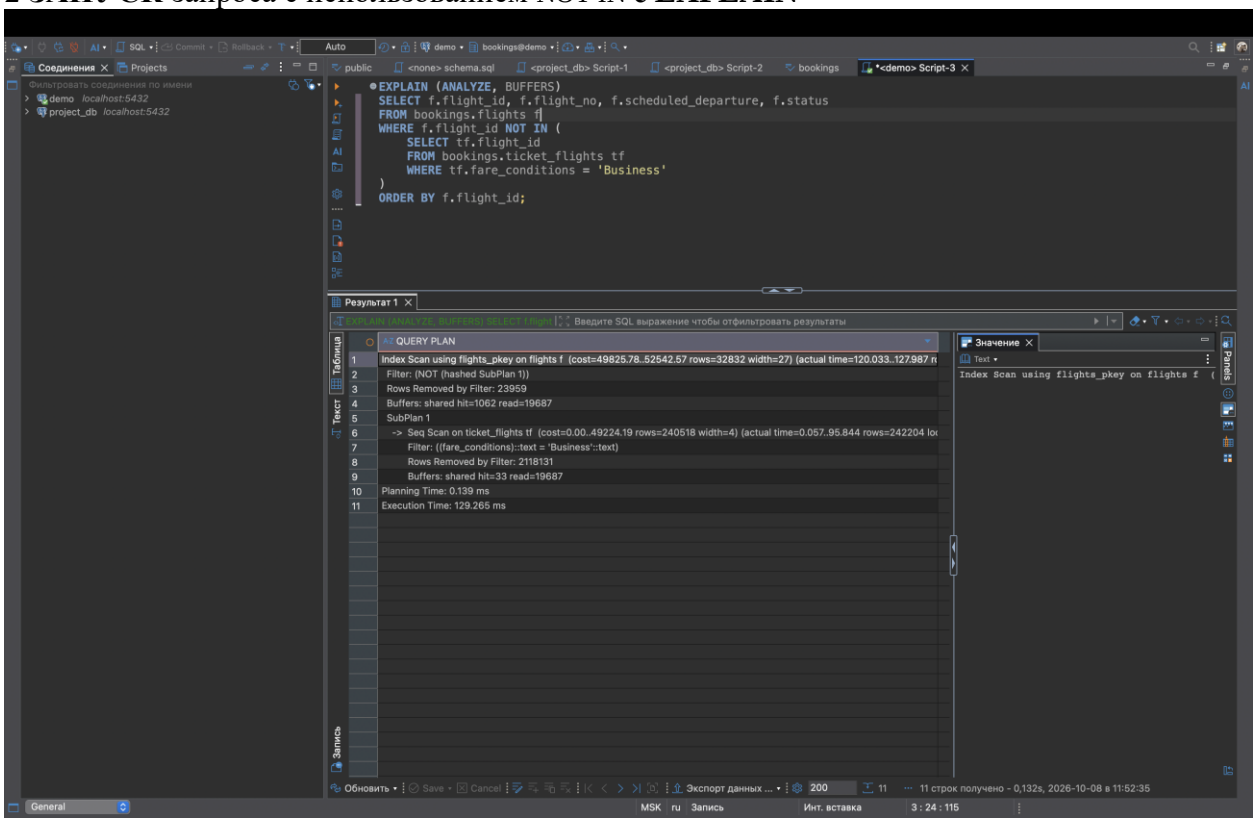
Простой ЗАПУСК запроса с использованием NOT IN



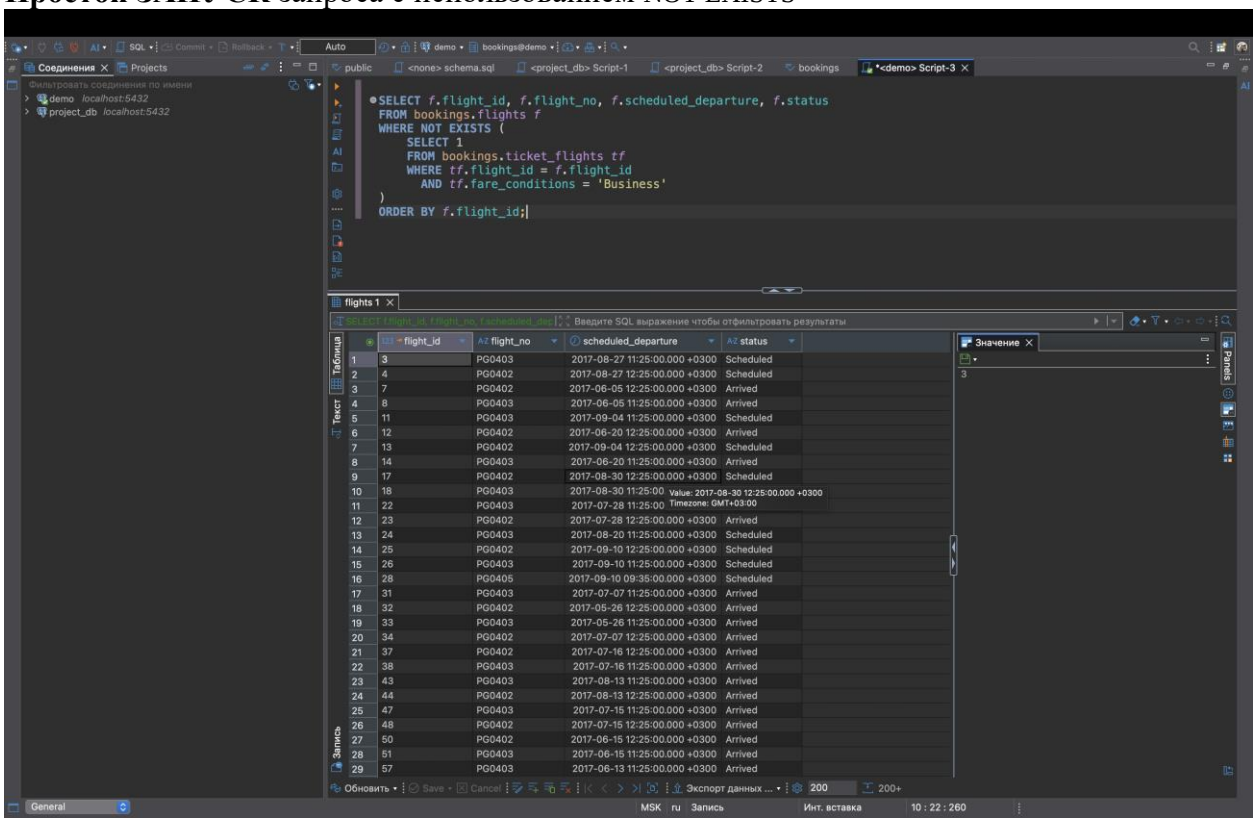
1 ЗАПУСК запроса с использованием NOT IN **с EXPLAIN**



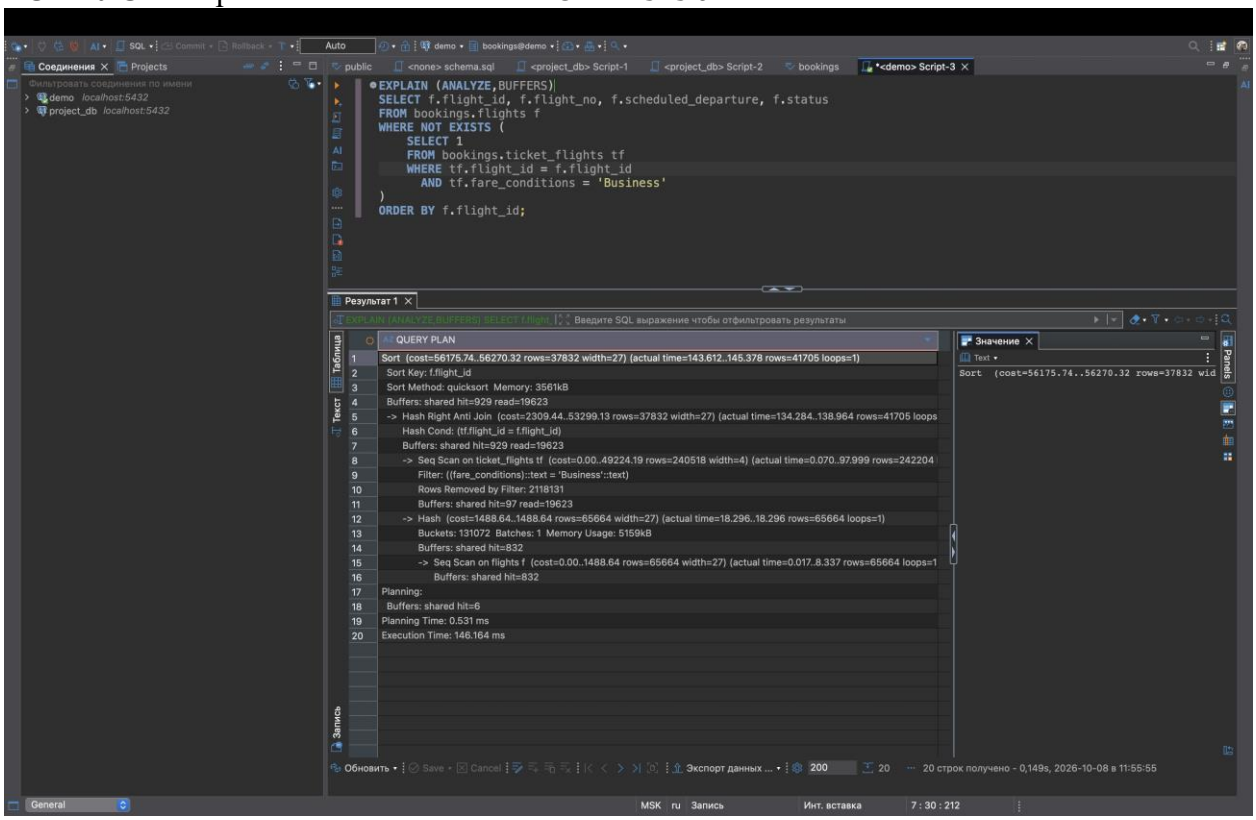
2 ЗАПУСК запроса с использованием NOT IN **с EXPLAIN**



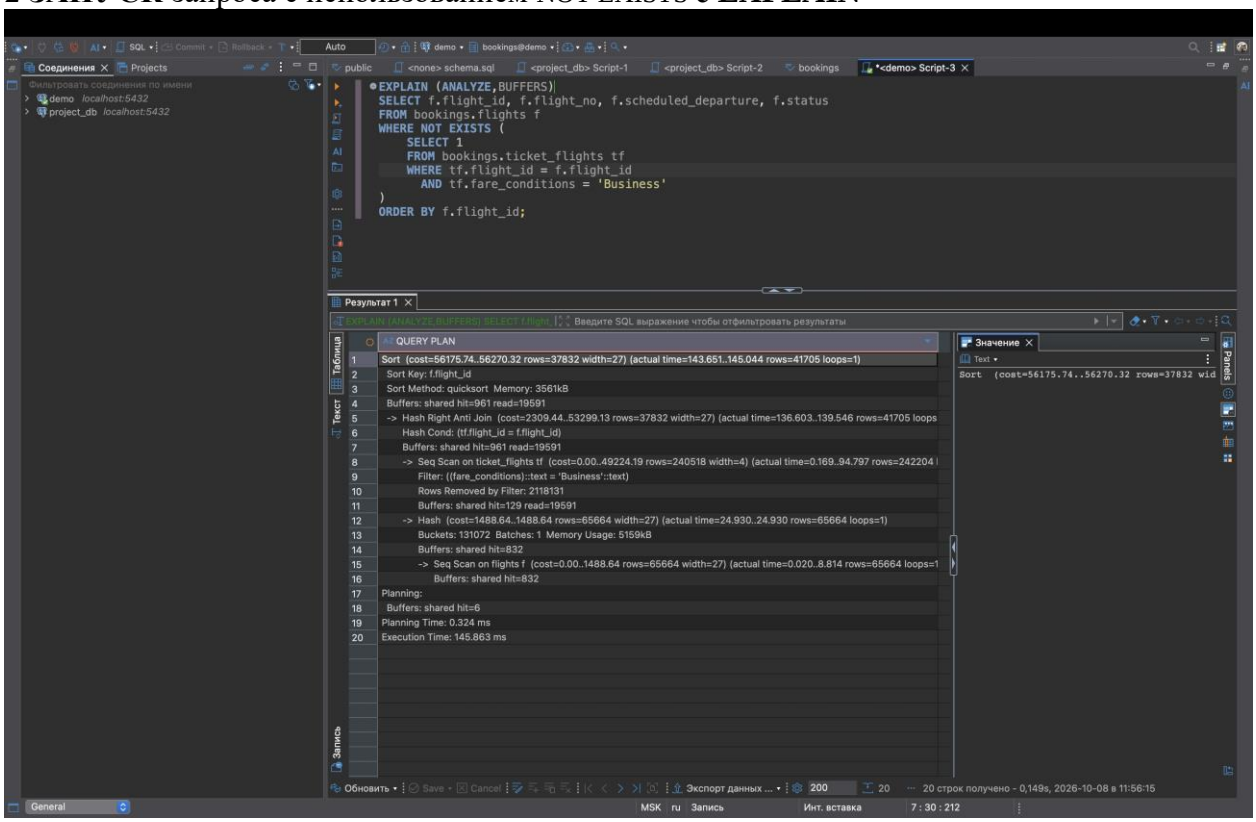
Простой ЗАПУСК запроса с использованием NOT EXISTS



1 ЗАПУСК запроса с использованием NOT EXISTS **с EXPLAIN**



2 ЗАПУСК запроса с использованием NOT EXISTS **с EXPLAIN**



ПОЯСНЕНИЕ к NOT IN:

При обоих запусках запрос читал данные мимо кеша PostgreSQL: ticket_flights читалась через Seq Scan (shared read=19720 при первом запуске и 19687 при втором), потому что большая таблица не удерживается в shared_buffers. Время выполнения снизилось с 157,2 до 129,3 мс, планирование с 0,45 до 0,14 мс. Подзапрос выполнился один раз как hashed SubPlan: Seq Scan по ticket_flights с фильтром fare_conditions = 'Business' отобрал 242 204 строки (отброшено 2 118 131) и положил их в хеш. Основной запрос шёл по индексу первичного ключа (Index Scan using flights_pkey), поэтому отдельной сортировки нет. Строки flights, чей flight_id есть в хеше, отбрасывались фильтром NOT (hashed SubPlan 1).

ПОЯСНЕНИЕ к NOT EXISTS:

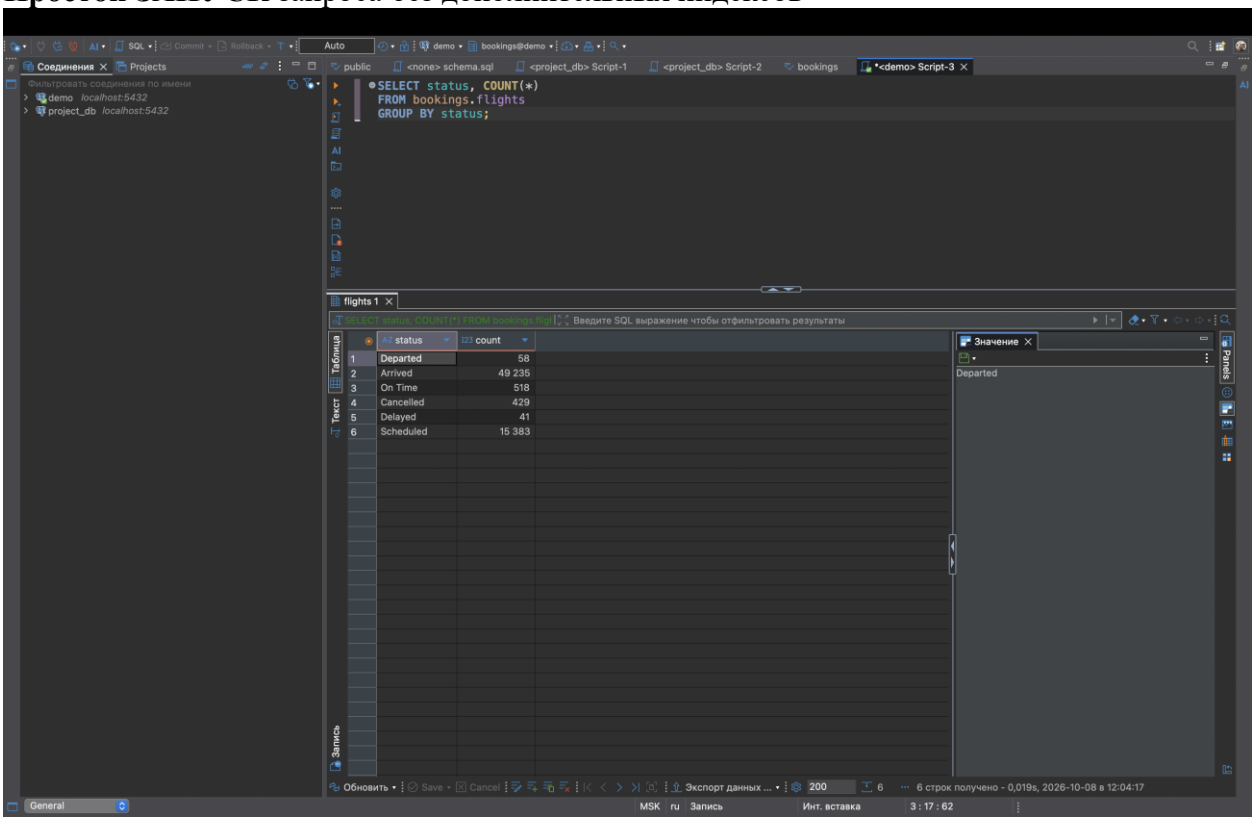
Запрос тоже читал ticket_flights мимо кеша PostgreSQL (shared read=19623 при первом запуске и 19591 при втором), а flights (832 страницы) целиком лежала в кеше (hit=832). Время выполнения почти не изменилось (146,2 и 145,9 мс), планирование снизилось с 0,53 до 0,32 мс. Планировщик построил Hash Right Anti Join: хеш-таблица строится по flights (65 664 строки, 5159 kB в памяти, один батч), а ticket_flights с фильтром Business проходит через неё один раз. Остаются рейсы без совпадений (41 705 строк). Сортировка по flight_id выполнена в памяти (quicksort, 3561 kB), временных файлов нет. Индексы для ticket_flights не использовались.

Почему NOT EXISTS эффективнее и чем опасен NOT IN:

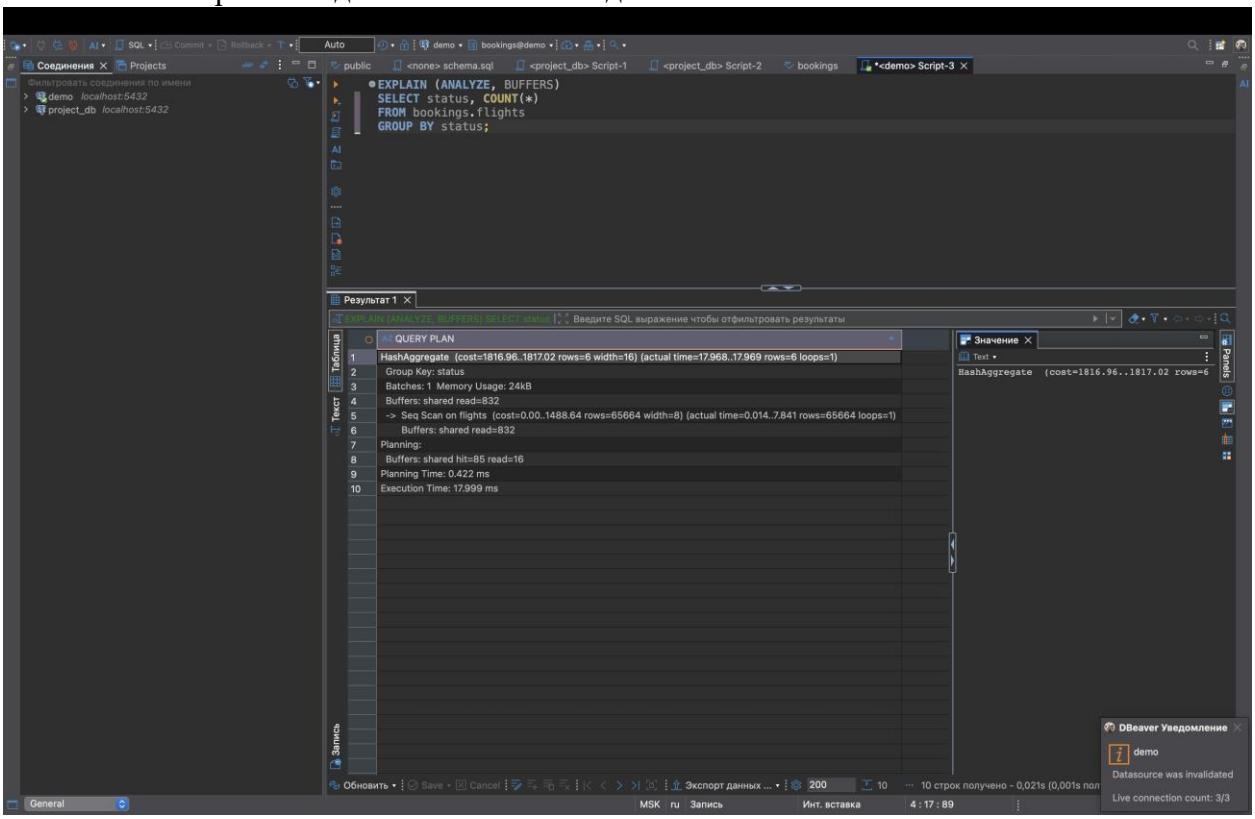
NOT EXISTS выражается как Anti Join: планировщик может выбрать любой алгоритм соединения и при первом найденном совпадении перестаёт искать для этой строки. Здесь он выбрал Right Anti Join, это тот же Anti Join, только хеш строится по другой таблице. На этих данных время обоих запросов почти одинаково (129–157 мс против 146 мс), поэтому выигрыш Anti Join здесь не проявился.NOT IN справился за счёт hashed SubPlan, который возможен, только если результат подзапроса помещается в work_mem. Гарантии такого плана нет: при большом подзапросе он превращается в SubPlan, который повторяется для каждой строки, и это на порядки медленнее. Anti Join для NOT IN планировщик построить не может из-за семантики NULL. x NOT IN (1, 2, NULL) раскрывается в x <> 1 AND x <> 2 AND x <> NULL, последнее даёт unknown, и условие никогда не станет истинным, поэтому один NULL в подзапросе обнуляет весь результат без ошибки. В ticket_flights.flight_id стоит NOT NULL, так что здесь оба запроса дают одинаковый результат, но NOT EXISTS безопасен при любых данных.

5. Индексное сканирование (Index Only Scan): Посчитайте количество рейсов для каждого статуса: SELECT status, COUNT(*) FROM flights GROUP BY status;

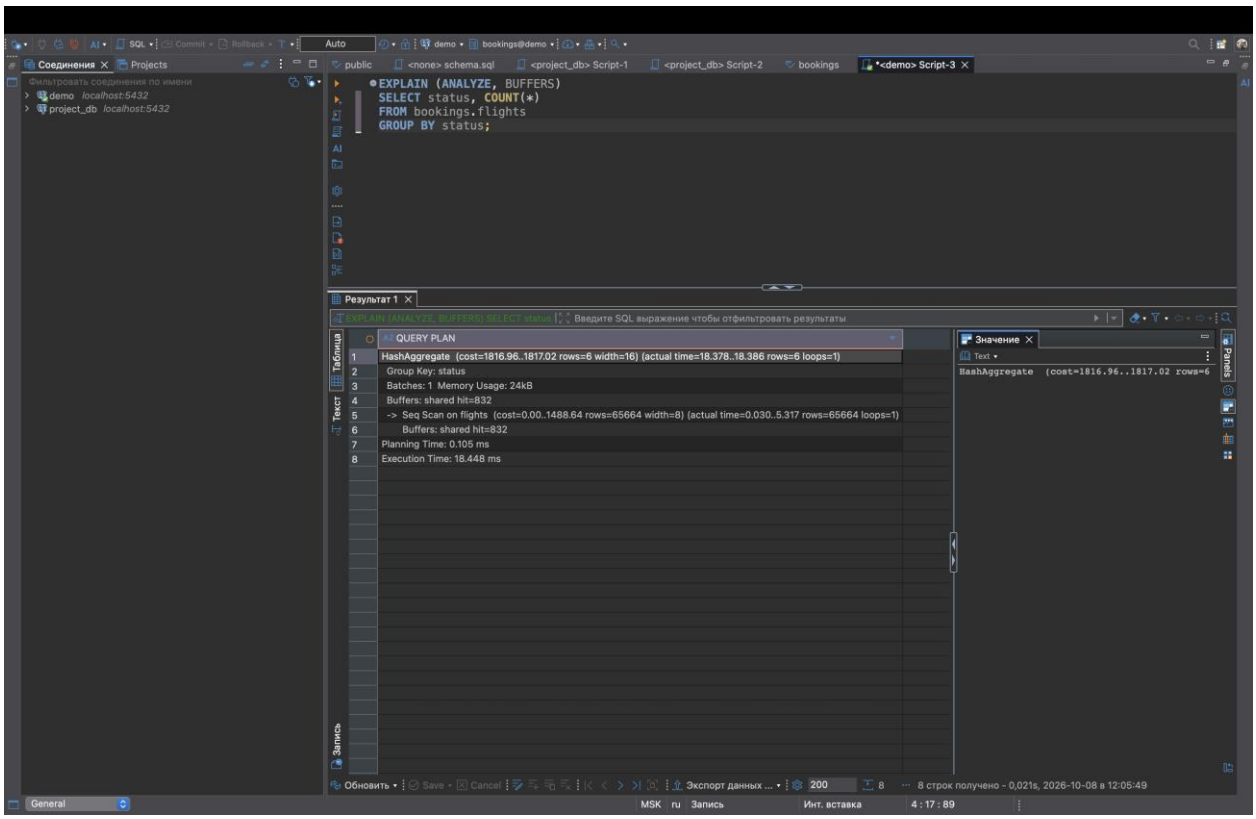
Простой ЗАПУСК запроса без дополнительных индексов



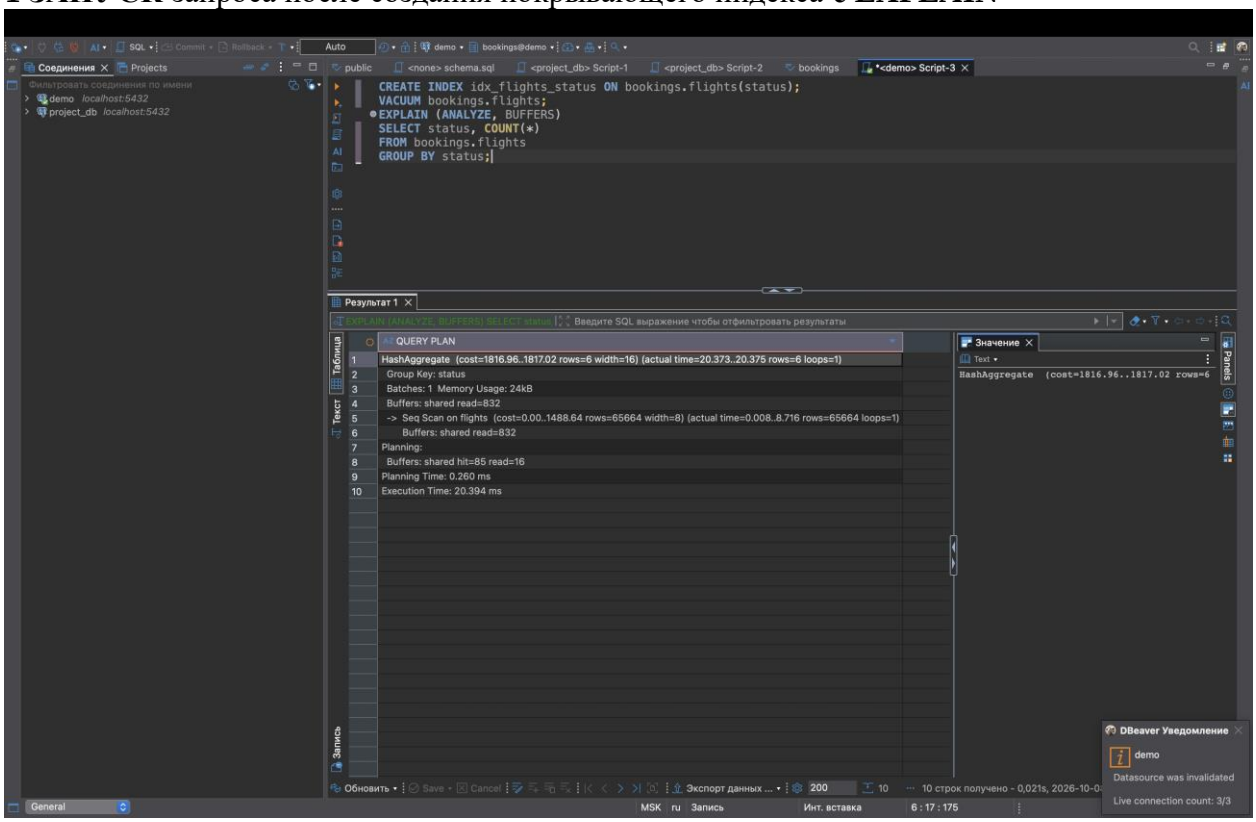
1 ЗАПУСК запроса без дополнительных индексов **с EXPLAIN**



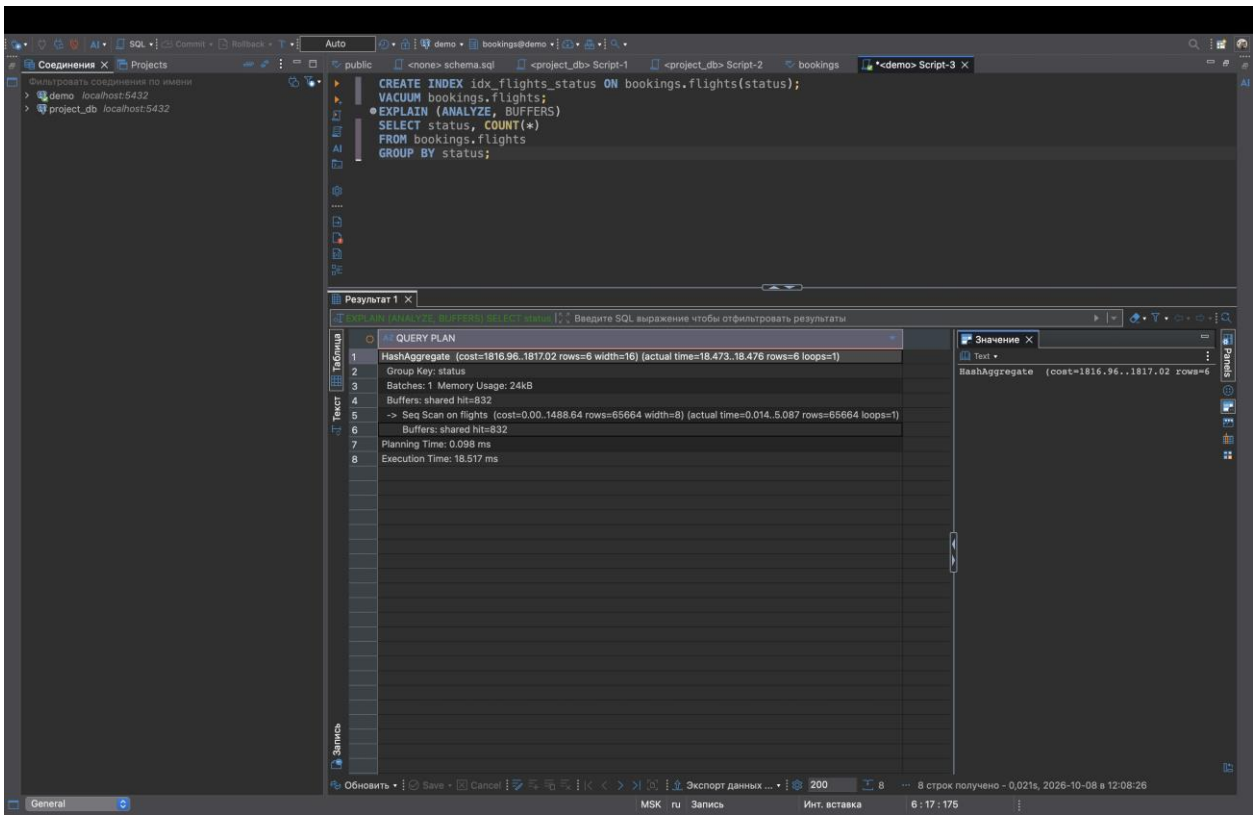
2 ЗАПУСК запроса без дополнительных индексов **с EXPLAIN**



1 ЗАПУСК запроса после создания покрывающего индекса **с EXPLAIN**



2 ЗАПУСК запроса после создания покрывающего индекса **с EXPLAIN**



ПОЯСНЕНИЕ (до индекса):

При первом запуске запрос работал с диском: flights читалась целиком (shared read=832), при втором все 832 страницы были в кеше (shared hit=832, read нет). Планирование снизилось с 0,42 до 0,11 мс, а выполнение почти не изменилось (18,0 и 18,4 мс), потому что основное время уходит на чтение и агрегацию 65 664 строк. Индекс не использовался: Seq Scan по flights и HashAggregate по status (6 групп, 24 kB в памяти, один батч, временных файлов нет).

ПОЯСНЕНИЕ (после создания индекса):

После выполнения CREATE INDEX idx_flights_status и VACUUM план запроса не изменился: планировщик по-прежнему использует Seq Scan по flights и HashAggregate по status (6 групп, 24 kB в памяти, временных файлов нет), а оценка стоимости осталась прежней (cost=1816.96..1817.02). При первом запуске запрос читал таблицу мимо кеша PostgreSQL (shared read=832, выполнение 20,4 мс), при втором все 832 страницы были в кеше (shared hit=832, read нет), выполнение 18,5 мс, планирование снизилось с 0,26 до 0,10 мс. Index Only Scan планировщик не выбрал, потому что запросу нужны все 65 664 строки таблицы, а различных значений status всего 6. Таблица маленькая (832 страницы), и её полное чтение оценено как не дороже обхода индекса, поэтому индекс на этих данных выигрыша не дал.

Что такое Index Only Scan и роль Visibility Map:

Index Only Scan означает, что данные читаются только из индекса, без обращения к таблице (heap). Индекс по status содержит всё, что нужно этому запросу, но не хранит информацию о видимости версий строк (MVCC). Поэтому обычно PostgreSQL вынужден заглядывать в таблицу и проверять, видна ли строка текущей транзакции. Visibility Map хранит для каждой страницы таблицы бит «все строки на этой странице видны всем транзакциям». Если бит установлен, проверку можно пропустить и взять значение прямо из индекса. Эти биты выставляет VACUUM, поэтому его и выполняют перед запросом. Число обращений в

таблицу показывает строка Heap Fetches: при актуальной карте оно близко к 0. Запрос с Index Only Scan быстрее, потому что индекс компактнее таблицы (хранит только status и указатель на строку), читается меньше страниц и нет случайных обращений к heap. На таблице такого размера разница небольшая, поэтому планировщик выбирает Seq Scan. На больших таблицах выигрыш становится заметным.